

Linear Secret-shared Shuffle with Malicious Security

Samuel Dittmer, Rohit Nema, and Rafail Ostrovsky

¹ Stealth Software Technologies, Inc.
`sam@stealthsoftwareinc.com`

² Department of Computer Science, Stanford University
`rnema@cs.stanford.edu`

³ Departments of Computer Science and Mathematics, UCLA
`rafail@cs.ucla.edu`

Abstract. Securely shuffling a secret-shared list is a vital sub-protocol in numerous applications, including secure sorting, secure list merging, secure graph processing, oblivious RAM, and anonymous broadcast. We demonstrate how to convert the folklore constant-round protocol for secure shuffling, which employs a delegated Fisher-Yates shuffle using rerandomizable encryption, into a maliciously secure constant-round protocol. This gives the first ever protocol that has a linear end-to-end time and communication for a two-party secret-shared shuffle with malicious security.

We prove the security of our protocol under the “linear targeted malleability” assumption on the homomorphic encryption system, as well as the natural assumptions of efficient ciphertext validity checks and rerandomizability. We also introduce a novel assumption, which we call *weak predicability*, and show that it is sufficient for security.

1 Introduction

Generating good randomness is an important problem in both theoretical and applied computer science. In classic card games, making sure that the deck is well-shuffled is necessary to ensure that the game is fair. In the context of multiparty computation, mutually untrusting parties often wish to shuffle data to ensure that the resulting protocol reveals nothing about the underlying data. These shuffles have valuable applications in other areas as a building block in oblivious RAM and anonymous broadcast constructions.

A “folklore” protocol for secure shuffle, see e.g. [27, 35, 43], works by using a rerandomizable encryption scheme to “delegate” the computation of the Fisher-Yates shuffle [29, 49] to one of the parties, and then repeat for each party. Fisher-Yates has complexity $\Theta(n)$, while there is an $\Omega(n \log n)$ lower bound on circuit-based protocols, so this bespoke protocol gives a $O(\log n)$ asymptotic speed-up over off-the-shelf techniques. This folklore protocol is not secure against malicious adversaries, who are not bound to perform the correct operations on the

encrypted shares. Recent works (e.g. [17, 24, 61], see below for more details) have improved on the $\theta(n \log n)$ cost of malicious-secure shuffling using off-the-shelf techniques, but still have a cost of $\omega(n)$ in at least one of online communication, online computation, or preprocessing.

Another approach to malicious shuffling would be to adapt techniques from verifiable computation to verify that the “delegated” party computes some permutation of the shared data. This is a form of RAM computation, using the insecure Fisher-Yates algorithm. Off-the-shelf approaches to verifiable computation for RAM require additional overhead, either through compilation to a circuit (with $\Omega(\log n)$ overhead, by a counting argument), through compilation to ORAM (with $\Omega(\log n)$ overhead [38, 50]), or through the use of SNARKs with support for RAM computations (with known approaches requiring $\Omega(\log n)$ overhead, see e.g. [23] for a discussion of recent work). Therefore, despite a series of recent advances in verifiable computation schemes using homomorphic encryption (e.g. [18, 31], see below in Section 1.3 for a more detailed discussion), these approaches are also unable to achieve linear end-to-end time.

1.1 Our Contributions

We give the first ever two-party secret-shared⁴ shuffling protocol that requires linear end-to-end time and communication. We prove it is secure against malicious adversaries given the existence of a linearly homomorphic encryption scheme satisfying the “linear targeted malleability” assumption of [8], with efficient ciphertext validity checks and rerandomizability. We also introduce a novel assumption, which we call *weak predicability*. Like “linear targeted malleability”, it is a weakening of “linear-only” encryption, but along a different dimension. We also prove security under this variant assumption.

Our protocol improves a key building block for constructing maliciously secure versions of vital cryptographic tools such as oblivious RAM and anonymous broadcast. Our approach gives a faster end-to-end time than off-the-shelf tools for verifiable computation under homomorphic encryption, which would require either running the shuffle computation as a circuit, or proving the correctness of a RAM computation, both of which would have $O(\log n)$ overhead.

1.2 Applications

The shuffle-sort paradigm of Hamada et al. [40] gave a general paradigm for oblivious sorting that showed how to use a shuffle step to compile any non-oblivious sorting protocol into an oblivious one. By first shuffling the list, information about the relative positions of the shuffled elements reveals nothing about the

⁴ Throughout this work, we focus on authenticated secret sharing as an abstraction and use the scheme in SPDZ [22] as an example for concreteness and measuring efficiency.

original list. This result was extended in [39] for the particular problem of *oblivious radix sort* using a similar “shuffle-reveal” paradigm. Works on the related problem of secure merge used the “tag-shuffle-reveal” paradigm for $O(n \log \log n)$ secure merge [27] and $O(n)$ secure merge [9, 16]. The “tags” for these secure merge protocols were additional metadata attached to the data elements that were shuffled and revealed along with the secret shared data. Oblivious sorting and merging are in turn fundamental building blocks in a range of applications such as oblivious RAM (introduced in [38], see, e.g. [26, 55, 62] for discussions of more recent treatments) and private information retrieval, surveyed in [60, 63]. Secure shuffles are also used for secure graph processing, including secure Dijkstra and MST graph processing [59].

1.3 Related Work

Semi-Honest Shuffles. In an insecure setting, the classic shuffling protocol is the Fisher-Yates shuffle [29], which performs shuffling with in-place swaps in $O(n)$ time. In multiparty computation, there are approximately three classical approaches: Circuit-based, permutation matrix-based, and homomorphic encryption based.

- A circuit-based approach to shuffling can use any off-the-shelf circuit approach to MPC (e.g. garbled circuits [66], GMW [37], BGW [6], see e.g. [25, 36] for more modern treatments of these protocols), along with a shuffling network, such as the Waksman network [64], which is of size $\Theta(n \log n)$. By a counting argument, there is a matching $\Omega(n \log n)$ lower bound for any swap-based approach. Many of these approaches can be made maliciously secure directly, using e.g., authenticated garbling (introduced in [65], see [54] for the most recent advances) or SPDZ-style authenticated MACs along with preprocessed authenticated triples [22].
- A permutation matrix based approach involves secret sharing a permutation matrix and then performing the shuffle using a single round of secure multiplication. This approach has $O(n^2)$ cost in communication and computation, and so may seem unappealing, but it is nevertheless useful as a building block in more complex protocols. This includes protocols in the online-offline model, which leads to solutions that have $O(n^2)$ computation and $O(n)$ communication in the online setting, and hybrid approaches where a circuit-based approach is used on blocks and then a permutation is used to shuffle each block [19].
- The homomorphic encryption approach is a folklore protocol, described in e.g. [27, 35, 43]. At a high level, it operates under a form of delegated computation: one party encrypts their share of a secret vector, allowing the other party to shuffle the encrypted pair locally with their own shares, and subsequently rerandomize to generate fresh encrypted shares of the other party. Our protocol is most closely related to this protocol, and we give a version of it expressed in our notation in Figure 1.

Online-Offline Malicious Shuffles. There are several shuffle protocols that achieve linear communication complexity in the online phase using some combination of a communication-computation trade-off and outsourcing preprocessing work to an offline model. Specifically, these works cannot achieve both linear communication and computation. One significant work in this area is the semi-honest protocol of Chase, Ghosh, and Poburinnaya [17], which offers an exponential trade-off between communication and computation. This work was improved to malicious security in [61]. See also [24, 51].

Three-Party Malicious Shuffles. In the multiparty case, many approaches are built on performing pairwise secure shuffles. In the case of an honest majority, ensuring correct behavior in the face of a malicious adversary can be easier than in the 2-party case, and shuffling can be accomplished by delegating to a series of committees. For example, in the three-party honest majority case, any two parties can shuffle the list by an agreed-upon permutation, using a check for malicious behavior. Performing three of these shuffles produces a list where the underlying permutation is unknown to all parties. We mention here Araki et al. [3], which presents one of the first linear time end-to-end multiparty shuffles in the literature, see also [1, 5, 57], which uses as the verification step a set-equality check that prefigures some of the techniques we use for verification. In the case of more algebraic approaches, we also mention among recent works e.g. [2, 19].

Verifiable Computation. In the last ten years, there has been considerable development in the study of performing private verifiable computation using fully homomorphic encryption. These schemes, however, still often impose requirements on the underlying FHE system, or have varied performance based on the problem.

Several works seek to support generic computations, applying some machinery on top of the homomorphic encryption to verify the correct computations are performed. Broadly, these techniques can be divided into three families of approaches: MAC-based, SNARK-based, and hardware-based. Hardware-based approaches use a trusted execution environment (TEE) to ensure that the correct homomorphic operations are applied; this approach to malicious security is orthogonal to our efforts here. We refer the interested reader to [48] for a more detailed survey of recent works in all three families of approaches, as well as improvements and optimizations to the SNARK-based approach of [31].

The MAC-based approach was pioneered by Gennaro and Wichs [34]. They called their approach *fully homomorphic message authenticators* (shortened by later authors to homomorphic MACs), and developed a system with verification work proportional to computation time. See also Catalano and Fiore [15] for a more concretely efficient approach for low-depth circuits requiring only one-way functions, with overhead polynomial in the *degree* of the circuit. Later works reduced the verifier’s computation to sublinear in the cost of plaintext circuit evaluation. Among these, we mention the work of Li et al. [52], which gives a scheme for verifying homogeneous polynomial computations that is efficient in

the amortized setting. Their scheme requires only the computation of one group exponentiation and four pairings per additional computation, but requires a semantically secure MAC that offers the same homomorphic operations as the FHE scheme.

The first SNARK built to support FHE, also known as verifiable FHE, was given by Fiore et al [28], although this scheme required homomorphic ciphertexts over $\mathbb{Z}_q$ for a large prime q . This was later improved in [10], which achieved more flexibility in the choice of FHE parameters, although both approaches are limited to FHE schemes that are compatible with the homomorphic hashing operation they use, such as the BV scheme. Another recent approach, Rinocchio [31], is a SNARK that is designed to prove statements about ring operations, allowing more efficient proofs of correctness of FHE operations which can be represented by expressions over an algebraic ring (such as Ring-LWE). This scheme, however, does not support relinearization, and so is limited in the functionalities that it can efficiently verify. The state-of-the-art verifiable FHE construction is [14], which works natively over rings and supports re-linearization.

Another category of works consist of essentially running a SNARK *inside* the FHE scheme. The first work introducing such a paradigm was [4]. However, they inherently suffer from lack of semantic security and public verifiability.

To summarize, these approaches can be used to get us a maliciously secure shuffle with linear communication, but the computation becomes $\Omega(n \log n)$ or greater since the verifiable computation must show that the a “shuffling” circuit is executed correctly, and such a circuit has at least $\Omega(n \log n)$ gates.

2 Technical Overview

We present four shuffling protocols in this paper. The first is the semi-honest folklore shuffle in Protocol 1, which we give for completeness and as a reference to indicate the techniques we need to add to gain malicious security. The second is a three-party information-theoretic shuffling protocol that is secure against malicious behavior by one corrupted party, which we give in Protocol 2. The protocol employs techniques similar to those in [3, 5], and we include it as a “warm-up” protocol, which demonstrates some of the techniques and proof structure we will use in our two-party protocol.

The third shuffling protocol is our main contribution, a two-party shuffling protocol using linearly homomorphic encryption that is maliciously secure, in Protocol 3. We give two proofs of security built on two different assumptions about the underlying homomorphic encryption system. The first proof is built on the “linear targeted malleability” assumption [8], which is a slight weakening of the “linear only” assumption to exclude oblivious ciphertext sampling. Linear targeted malleability and linear-only encryption have been used to build SNARGs [11, 12] and SNARKs [20, 32, 33, 44]. The second assumption is a novel one, which we call the “weakly-predicable” assumption. Informally, it limits the sort of transformations on ciphertexts that an adversary can perform to those

that test “weak predicates”. In Section 6, we prove it is also weaker than “linear only” encryption, and is in fact sufficient to establish the security of our protocol.

Finally, we give a variant of our three-party shuffle in which “delegated” shares are delegated without their corresponding authenticated shares, in Protocol 4. Because the shares are “delegated” in an unauthenticated fashion, additional checks are required to confirm that the correct operations are performed. Although this makes the protocol more complex, it does not alter the fundamental approach. We describe this in more detail in Section 7, and explain how our two-party shuffling protocol could similarly be altered to omit the homomorphic encryption of the authenticated shares, leading to potential efficiency improvements. In addition to not requiring communication of half as many ciphertexts, this variant demonstrates that the approach presented in this paper is fundamentally different from the standard compilation of honest-but-curious to malicious blueprints, which require authenticated shares to protect each computed value.

We proceed by first providing a high-level overview of the building blocks in Section 2.1. Then, in Section 2.2 and Section 2.3, we show how to glue our techniques to obtain a maliciously secure 3-party and 2-party shuffle respectively.

2.1 Building Blocks

We present the folklore 2-party semi-honest secure half-shuffle protocol for additive secret shares using linearly homomorphic encryption (LHE) in Protocol 1. (See Section 3 for preliminaries on LHE and secret sharing.) Briefly, P_1 encrypts its share of the input vector and sends it to P_2 who first re-randomizes both (1) the encrypted (additive) share by adding an encryption of a random vector and subtracting the same random vector from its plaintext share⁵, and (2) the ciphertext itself⁶; and second, shuffles them and sends it back to P_1 .

Protocol 1 2-party Semi-honest Secure Half-Shuffle (Folklore)

Input: secret shares of a vector $\mathbf{v} \in \mathbb{F}^n$, $[\![\mathbf{v}]\!]$; P_2 with permutation $\pi \leftarrow S_n$.

Output: $[\![\pi(\mathbf{v})]\!]$, secret shares of permuted vector $\pi(\mathbf{v})$.

- 1: P_1 generates an LHE key-pair $(pk, sk) \leftarrow \text{LHE.KeyGen}(1^\kappa)$ and publishes pk .
 - 2: P_1 encrypts $[\![\mathbf{v}]\!]_1$, $C_v \leftarrow \text{LHE.Enc}(pk, [\![\mathbf{v}]\!]_1)$, and sends C_v to P_2 .
 - 3: P_2 samples uniformly random mask, $\mathbf{r} \xleftarrow{\$} \mathbb{F}^n$ and homomorphically computes, $C_v^\pi \leftarrow \pi(C_v) \odot \text{LHE.Enc}(pk, \mathbf{r})$, and sends C_v^π to P_1 .
 - 4: P_1 decrypts and sets, $[\![\mathbf{v}^\pi]\!]_2 := \text{LHE.Dec}(sk, C_v^\pi)$
 - 5: P_2 sets $[\![\mathbf{v}^\pi]\!]_1 := \pi([\![\mathbf{v}]\!]_2) - \mathbf{r}$.
 - 6: Output $[\![\mathbf{v}^\pi]\!]$.
-

⁵ or equivalently, adding a secret-sharing of the zero vector with P_1 ’s share encrypted.

⁶ We assume our linearly homomorphic encryption scheme supports re-randomization. (See Section 3.2 for more details.) To our knowledge, every homomorphic encryption scheme supports re-randomization e.g. by homomorphically evaluating the ciphertext with a fresh ciphertext of 0 or 1. For brevity, we assume re-randomization is performed after every homomorphic evaluation.

We outline the techniques we use to extend the folklore protocol to obtain malicious security in both the 2-party and 3-party setting.

Shuffle from half-shuffle. Throughout this paper, we focus our attention on one-sided or “half” shuffles, where one or more parties know the permutation being applied, and the other party does not. By applying the half-shuffle twice with the roles of the parties reversed, one immediately obtains an oblivious shuffling protocol. Building from half-shuffles has been done by most prior works on oblivious shuffling (e.g. [17, 24, 61]).

Authenticated shares. We use authenticated secret sharing to represent the inputs and outputs as in the SPDZ framework [22]. Specifically, we use the authentication to verify correctness of the output and prevent deviation.

The novel idea in our work is to show how to blend (linearly) homomorphic encryption and authenticated secret shares. Our technique involves sending encrypted shares to the other party without compromising security. The homomorphic cryptosystem enables us to perform one-sided computation on shares (which in our case is a half-shuffle). In fact, this does not prove trivial. Several subtleties arise and are subsequently dealt with to avoid selective failure attacks for example. We elaborate in Section 2.4.

Emulating HE with three parties. Our three party protocol is built on the observation that homomorphic encryption can be *emulated* by delegating computation to two parties—one of whom is honest—and having them authenticate that they are performing the correct operations. As long as the two delegated parties are given authenticated values, their “emulated” homomorphic encryption in fact satisfies a much stronger condition than the “linear targeted malleability” assumption. Under the “linear targeted malleability” assumption, the delegated party can tamper with the provided ciphertexts but is restricted to tampering that produces affine transformations on the underlying plaintexts, or obviously sampled ciphertexts where the plaintext is unknown. In the three party setting, *any* deviation from semi-honest behavior by either of the delegated parties will cause the authentication checks to fail.

In Section 7, we give a variant of our three party shuffle where the two delegated parties are instead given unauthenticated messages. This variant requires additional techniques to check the correctness of their delegated shares against their original 3-way authenticated shares; informally, our three party shuffle with unauthenticated resharing is somewhere between our three party shuffle with resharing and our two party shuffle with homomorphic encryption in terms of protocol complexity.

Set equality. We use a protocol for set equality check that runs in linear-time, see e.g. [30]. For two sets of size n , $A := \{a_i\}_{i \in [n]}$, $B := \{b_i\}_{i \in [n]}$, where $a_i, b_i \in \mathbb{F}$,

the protocol is built on evaluating the polynomial,

$$f(X) := \prod_{i=1}^n (X - a_i) - \prod_{i=1}^n (X - b_i)$$

at a random point $r \xleftarrow{\$} \mathbb{F}$. This polynomial is the zero polynomial if and only if $A = B$. If $f(X) \neq 0$, then by the Schwartz-Zippel lemma [56], $f(r) = 0$ with probability at most $n/|\mathbb{F}|$. Since n is polynomial in λ and $\mathbb{F}$ is chosen such that $|\mathbb{F}| = \Omega(2^\lambda)$, this is negligible. Therefore, it is sound to check if $f(r) = 0$.

The test also works on top of authenticated shares with n secure multiplications and an authenticated sharing of r .

Note that in the setting of proof systems, the verifier is allowed to learn information on A and B if the prover is lying about $A \stackrel{?}{=} B$; however, this would lead to a selective failure attack in our setting. Observe that if r is not known to either party, then information on A and B remains hidden. The preprocessing we introduce in Section 3.1 indeed provides the functionality to produce authenticated shares of uniformly random values not known to either party.

Let $\mathcal{F}_{\text{set-eq}}$ denote the ideal functionality for the polynomial test described above on authenticated shares which takes as input 2 vectors of authenticated shares and outputs an authenticated sharing of 0 if the vectors are equal as sets and (random non-zero otherwise).

2.2 Warm-up: Information-theoretic 3-party Shuffle

The core idea of our information-theoretic 3-party shuffle is to transform the folklore protocol using the emulation of HE with three parties described above. In the folklore protocol (Protocol 1), P_1 delegates the shuffling procedure over the shares to P_2 . To ensure P_2 doesn't learn the underlying value, P_1 simply encrypts its shares. P_2 also rerandomizes the shares to ensure P_1 doesn't learn the permutation. Comparing Protocol 1 to our three-party protocol (Protocol 2), we can see that the LHE.Enc operation has been replaced by P_1 re-sharing their shares with P_2, P_3 , and the LHE.Dec operation has been replaced by P_2 and P_3 re-sharing back to P_1 .

We assume the authenticated secret sharing scheme supports authenticated re-sharing. We capture this with the Authenticated Secret Sharing (AuSS) ideal functionality in Fig. 1. SPDZ is our canonical example of an AuSS scheme, but we introduce some additional commands we need in our arguments. In particular, we note that to ensure that the re-sharing of AuSS is correct, an implementation in SPDZ would require an authentication check step as part of the re-sharing. (See Section 3.1.)

Therefore, P_2 and P_3 have no opportunities for foul play. Any operation that P_2 (or P_3) performs that is different from applying the desired permutation will cause the authenticated re-sharing to fail (e.g. in SPDZ, it will break the MAC check). For P_1 , however, the use of AuSS may provide a new opportunity to make the probability of abort input-dependent, which introduces a *selective*

failure attack. For example, by adding a random corrupting term to an entry at index i of their initial shares, and subtracting the same term from position j of their output shares, P_1 can test whether $\pi(i) = j$; however such an attack will also cause authenticated re-sharing to fail. We give the full protocol and security proofs in Section 4.

In our variant three-party shuffle, given in Protocol 4, P_1 sends re-shared shares to P_2 and P_3 without authentication. This protocol still uses AuSS to check the consistency of the inputs and outputs, but does not use AuSS in the process of delegating and reconstructing shares. In the context of SPDZ, this means we can send half as many values during the delegation and reconstruction, at the cost of additional checks.

We replace our use of the generic authentication check to verify the correctness of P_1 's re-sharing with a simple challenge-response test where P_2 and P_3 perform an unknown affine transformation on P_1 's delegated shares. We now also have to guard against any of the three parties choosing to reconstruct their shares incorrectly, and authenticating the wrong values. We do this by performing a bespoke set equality check that relies on the fact that both P_2 and P_3 have knowledge of the permutation π . See Section 7 for more details.

2.3 2-party Shuffle using Linearly Homomorphic Encryption

As in the three-party case, we build off of the two-party folklore half shuffle in Protocol 1, and impose additional checks to achieve *security with abort*. We again use an AuSS scheme, both to authenticate the inputs and outputs, and to perform tests of correct behavior along the way. In the context of SPDZ AuSS, when P_1 delegates their shares to P_2 using homomorphic encryption, they encrypt their shares along with their MAC shares. We stress that this is not the same as the homomorphic MACs used in e.g. [15, 18, 34, 52], since there is no guarantee that P_1 encrypts the correct MAC shares, or that P_2 performs consistent operations on the MACs.

There are two further complications we must address in the two-party shuffle. First, because P_1 sends encrypted shares to P_2 , P_2 cannot perform the authentication check generically on the delegated shares. We instead use a challenge-response test here similar to the test used in our variant three-party shuffle where authentication is not used.

Second, because P_2 has access to ciphertexts encrypted under a linearly homomorphic encryption scheme, they can perform a broad range of malicious transformations before returning the allegedly shuffled shares, giving a broad range of selective failure attacks. As in the variant three-party shuffle, we use set-equality checks to verify correctness, but to address the selective failure attacks we also have P_1 *split* their shares using an authenticated mask, encrypting them under two different public keys.

Our resulting protocol blends authenticated shares and linearly homomorphic encryption by replacing the machinery of the homomorphic MAC approach with bespoke lightweight correlation checks to instantiate a maliciously secure two-

party shuffle. We decompose the protocol (presented in Protocol 3) into four parts,

1. **Commit to permutation.** First, P_2 “commits” to their permutation π by generating an authenticated sharing of $\pi(\mathbf{z})$ for random $\mathbf{z} \xleftarrow{\$} \mathbb{F}^n$. Doing so ensures the permutation is independent of the messages sent by P_1 , which becomes necessary when proving security against malicious P_2 .
2. **Split and encrypt shares.** P_1 will *delegate* applying the permutation to P_2 by sending shares encrypted using an LHE scheme. To prevent selective tampering, P_1 first *splits* the shares and encrypts to each split using an independent encryption key. We prove that P_2 can no longer selectively tamper under two different assumptions of the underlying LHE scheme. It is however still possible for P_1 to selectively tamper with its ciphertext. We thwart such attacks (and any other) and enforce consistency between the authenticated shares of the input and the ciphertext by performing a challenge-response test using an unknown affine transformation, similar to our approach in the variant three-party shuffle.
3. **Permute and rerandomize shares.** P_2 proceeds to apply π to both the encrypted shares of P_1 and its own. To prevent leakage of π , P_2 *re-randomizes* the shares. Since P_1 ’s shares are encrypted, this is only possible if the AuSS scheme supports linear re-randomization. To our knowledge, any AuSS such as the one in SPDZ supports this by construction. (See Section 3.1.)
4. **Check consistency of shares.** Finally, we must ensure that P_2 indeed permuted the shares and the permutation was applied consistently on both splits of P_1 ’s shares, and the initial commitment. We do so using the set equality check described in Section 2.1.

We present and prove security of our final construction in Section 5.

2.4 Gaining Intuition: Attacks and Assumptions

Motivation. Off-the-shelf tools for transforming homomorphic encryption schemes into verifiable computation can support arbitrary computations, but their requirements or performance are tied to the choice of underlying homomorphic encryption scheme, as discussed in Section 1.3. Our approach, in contrast, is designed for a particular problem, but aims for minimal assumptions on the underlying homomorphic encryption scheme.

The advantage of our approach is that it removes the $O(\log n)$ overhead from transforming the shuffling algorithm into a circuit or verifying that the correct RAM computation was performed. The challenge is the broad family of attacks that arise when individual computations are not verified. This is why we require *some* assumptions on our choice of homomorphic encryption scheme.

Attacks. As a simple example of a selective failure attack that can arise under our approach, if one party holds encryptions $\text{Enc}(v), \text{Enc}(w)$ and wishes to test whether $v = w$, they can choose a random value r and compute $\text{Enc}(r(v - w))$

under a linearly homomorphic encryption scheme, and add this term to future messages they send. This will almost surely cause the post-conditions to fail unless $v = w$.

To thwart some of these attacks, we replace $\text{Enc}(v)$ with $(\text{Enc}_1(a), \text{Enc}_2(m))$, where $a + m = v$ is an additive sharing of v and Enc_1 and Enc_2 represent encryption under two distinct public keys. However, this does not remove all attacks. Suppose that the plaintext space is $\mathbb{Z}/p\mathbb{Z}$. Under fully homomorphic encryption, a malicious party can compute the Legendre symbol $a^{(p-1)/2}$ and determine the quadratic residuosity of both a and m . Taking advantage of the structure of $\mathbb{Z}/p\mathbb{Z}$, the malicious party then chooses nonzero r_1, r_2 and computes $(\text{Enc}_1(r_1 - r_1 a^{(p-1)/2}), \text{Enc}_2(r_2 + r_2 (-m)^{(p-1)/2}))$. This expression is equal to $(\text{Enc}_1(0), \text{Enc}_2(0))$ with probability $\frac{1}{4}$ if p does not divide $v = a + m$, but it is never equal to $(\text{Enc}_1(0), \text{Enc}_2(0))$ if $p|v$. This leads to a selective failure attack where the malicious party adds $\text{Enc}_1(r_1 - r_1 a^{(p-1)/2})$ and $\text{Enc}_2(r_2 + r_2 (-m)^{(p-1)/2})$ to two separate messages.

Linear Targeted Malleability. At this point, instead of attempting further ad hoc patches, we impose an assumption on the homomorphic encryption system to reduce the sorts of attacks that are available. In Section 5, we restrict ourselves to LHE schemes that satisfy the “linear targeted malleability” assumption, which informally states that all an adversary can do with a linearly homomorphic encryption system is linear operations, or replacing a ciphertext with an obviously sampled ciphertext corresponding to an unknown plaintext. This closes the selective failure attacks outlined above because the only transformations of vectors $\mathbf{a}, \mathbf{m}$ are linear, the kernels of any nontrivial transformation will be a proper subspace of $\mathbb{F}^n$, and with high probability a randomly chosen pair $(\mathbf{a}, \mathbf{m})$ with $\mathbf{a} + \mathbf{m} = \mathbf{v}$ will not lie on any fixed subspace.

The linear targeted malleability assumption was introduced in [8] as a way of compiling LIPs (Linear Interactive Proofs) into SNARKs. This assumption, or the stronger assumption of linear-only encryption (which removes the possibility of obviously sampled ciphertexts), has been used to develop many modern SNARKs schemes (see e.g. [20, 44]). It is a non-falsifiable assumption, but it is nevertheless considered useful, because attacks on it may lead to more efficient ways to transform a linearly homomorphic encryption system into a fully homomorphic encryption system.

Weakly Predicable Encryption. In this work, we introduce a notion of “weakly predicable” encryption. This is (as far as we know) a novel assumption, but we prove that it is weaker than the linear-only assumption, and also sufficient to establish security against malicious adversaries. Informally, it is a more dramatic weakening of “linear only” encryption, since it supports, for example, polynomials of bounded degree, but it appears unlikely that it is directly comparable to the linear targeted malleability assumption, due to different roles the auxiliary randomness plays in the definitions.

If we return to the example above of a selective failure attack based on quadratic residuosity, the origin of the attack is that the malicious party can test two predicates $\left(\frac{a}{p}\right) \stackrel{?}{=} 1$ and $\left(\frac{-m}{p}\right) \stackrel{?}{=} -1$, and the distribution of those two predicates is related to the predicate $a + m \stackrel{?}{=} 0$. We call the predicate $\left(\frac{a}{p}\right) \stackrel{?}{=} 1$ a *strong predicate*, which means (informally) that it distinguishes a set of inputs representing a constant fraction of the domain. A linear transformation on $\mathbb{F}^n$, with $|\mathbb{F}| > 2^\lambda$, in contrast, is a *weak predicate*, because it can only be used to distinguish elements in its kernel, which is a negligible fraction of its domain. An encryption system has the *weakly predicable* property, then, if it can only be used to test weak predicates. This includes, for example, a somewhat homomorphic encryption system that supports some bounded depth of multiplications, since the predicates correspond to zero sets of polynomial expressions of bounded degree, which also cover a negligible fraction of the domain. We give formal definitions and proofs in Section 6.

3 Preliminaries

Fix a finite field $\mathbb{F}$ of prime order p . Let λ be the security parameter. For security, we assume $p = \Omega(2^\lambda)$.

Sets. For any natural number n , denote by $[n]$, the set $\{1, \dots, n\}$. We write $a \stackrel{\$}{\leftarrow} A$ when a is sampled *uniformly at random* from the set A .

Secret sharing. For $x \in \mathbb{F}$, denote $\llbracket x \rrbracket$ as the additive secret sharing of x . Let $\llbracket x \rrbracket_j$ be P_j 's additive share of x . Similarly, denote $\langle x \rangle$ as an authenticated secret sharing (AuSS) of x and let $\langle x \rangle_j$ be P_j 's authenticated share of x .

Vectors. We use boldface for vectors, e.g., $\mathbf{v}, \mathbf{a}, \mathbf{m}, \dots$ and denote the i th component of $\mathbf{v} \in \mathbb{F}^n$ as $\mathbf{v}[i]$.

Operating on vectors. We overload notation when operating on vectors. Any operation performed on vectors is performed component-wise. For example, we denote the additive/authenticated secret sharing of a vector, $\mathbf{v} \in \mathbb{F}^n$ as the vector of its component-wise secret sharing, $\llbracket \mathbf{v} \rrbracket = (\llbracket \mathbf{v}[1] \rrbracket, \dots, \llbracket \mathbf{v}[n] \rrbracket)$ and similarly for $\langle \mathbf{v} \rangle$. Similarly, for an encryption algorithm Enc , $\text{Enc}(\mathbf{v}) = (\text{Enc}(\mathbf{v}[1]), \dots, \text{Enc}(\mathbf{v}[n]))$.

Randomness. For any PPT algorithm A , denote the distribution of its internal randomness by $\text{coins}(A)$.

3.1 Authenticated Secret Sharing and the SPDZ Framework

In this section, we introduce abstractions of authenticated secret sharing (AuSS) and a preprocessing step as ideal functionalities. We describe a concrete implementation of AuSS using SPDZ.

The SPDZ framework [22, 45] enables computation of general arithmetic circuits in MPC with security *with abort* against active/malicious adversaries. A SPDZ computation is split into two phases, (1) an input-independent preprocessing phase wherein it samples correlated randomness, followed by (2) an on-line phase that consumes the randomness and computes over the inputs. At a high level, SPDZ uses (additive) authenticated secret sharing implemented using information-theoretic Message Authentication Codes (IT-MACs) [7, 22]. Informally, the MAC prevents deviating from the protocol by performing a consistency check between the output shares and its MAC at the end.

In this work, we will abstractly define authenticated secret sharing as a black box ideal functionality which encompasses SPDZ’s implementation of authenticated secret sharing. We also describe an abstraction of the SPDZ preprocessing functionality below with respect to the abstract authenticated secret sharing scheme.

Authenticated Secret Sharing (AuSS). For generality, we black-box the AuSS functionality as an ideal functionality, $\mathcal{F}_{\text{AuSS}}$ in Figure 1. In particular, $\mathcal{F}_{\text{AuSS}}$ can perform addition, multiplication and scaling; open shares (of the value); perform authentication checks (to verify the consistency of a share); re-randomize existing authenticated secret shares; and reshare a secret share to subsets of parties.

Throughout this work, our functionalities keep track of authenticated secret shares using id’s or “handles” maintained as state in a dictionary called Auth. We assume that any call to a functionality which takes authenticated secret shares as input first goes through an implicit translation from shares to their respective id’s. We do not explicitly elaborate in our constructions as is common for brevity.

Preprocessing. SPDZ uses an input-independent offline preprocessing phase before running online on the inputs. The preprocessing generates the necessary correlated randomness for computation including shares of the symmetric MAC key, authenticated randomness, and Beaver triples in the concrete SPDZ implementation. It is useful to capture the preprocessing functionality separately as due to its modularity, we can reason about the online phase independently if we assume the preprocessing generates sufficient correlated randomness.

We abstract the SPDZ preprocessing as an *ideal functionality* denoted by $\mathcal{F}_{\text{PREP}}$ which, in this work, needs to only support the following sub-functionalities:

1. Generate random values: sample random values $r \xleftarrow{\$} \mathbb{F}$,
2. Generate authenticated random values: sample a random value $r \xleftarrow{\$} \mathbb{F}$ and output its authenticated secret share $\langle r \rangle$,
3. Generate Beaver triples: output authenticated shares of Beaver triples used to compute multiplication of authenticated shares.

$\mathcal{F}_{\text{PREP}}$ has indeed been realized for the SPDZ AuSS. For brevity, we do not re-specify the functionality formally as it has been presented in several prior

Functionality $\mathcal{F}_{\text{AuSS}}$

State. A dictionary Auth to map identifiers to authenticated values in $\mathbb{F}$. $\mathcal{F}_{\text{AuSS}}$ supports the following commands/sub-functionalities. For each command, if the identifier does not exist, send **abort** and halt. Unless otherwise noted, the functionality waits to receive the command from every party in the system and each identity is concatenated with the set of parties with which it is shared with the default set of every party which we omit for brevity.

1. **Create**(x, id_x): store $\text{Auth}[\text{id}_x] \leftarrow x$.
2. **Add**($\text{id}_x, \text{id}_y, \text{id}_z$): store $\text{Auth}[\text{id}_z] \leftarrow \text{Auth}[\text{id}_x] + \text{Auth}[\text{id}_y]$. (We denote this using the binary operator “+.”)
3. **Mult**($\text{id}_x, \text{id}_y, \text{id}_z$): store $\text{Auth}[\text{id}_z] \leftarrow \text{Auth}[\text{id}_x] \cdot \text{Auth}[\text{id}_y]$. (We denote this using the binary operator “.”)
4. **Scale**($\text{id}_x, c, \text{id}_y$): store $\text{Auth}[\text{id}_y] \leftarrow c \cdot \text{Auth}[\text{id}_x]$. (We denote this using the overloaded binary operator “.”)
5. **Open**(id_x): return $\text{Auth}[\text{id}_x]$.
6. **Check**(id_x): return 1 if $\text{Auth}[\text{id}_x]$ is set, else 0.
7. **ReRand**(id_x, id_y): store $\text{Auth}[\text{id}_y] \leftarrow \text{Auth}[\text{id}_x]$.
8. **ReShare**($S_1, \text{id}_x, S_2, \text{id}_y$): upon receiving id_x from each party in S_1 and id_y from each party in S_2 , store $\text{Auth}[\text{id}_y, S_2] \leftarrow \text{Auth}[\text{id}_x, S_1]$.

Fig. 1. Ideal functionality for Authenticated Secret Sharing, $\mathcal{F}_{\text{AuSS}}$

works [21, 22, 46]. In practice, preprocessing is implemented using Somewhat Homomorphic Encryption (SHE) and Oblivious Transfer (OT) [22, 41, 45, 47, 58]. The most expensive cost is to generate the Beaver triples. Roughly, the preprocessing cost is asymptotically linear in the number of required multiplications. Lastly, we assume that any implementation of $\mathcal{F}_{\text{PREP}}$ can be simulated to extract the underlying randomness used. This is true for the SPDZ preprocessing for example (e.g. as described in [61]).

We outline more details about the SPDZ framework below to describe the SPDZ implementation of $\mathcal{F}_{\text{AuSS}}$ for concreteness.

IT-MACs. The SPDZ preprocessing samples $\alpha \xleftarrow{\$} \mathbb{F}$ as the global MAC key. The MAC of a value $x \in \mathbb{F}$ is $\gamma(x) := \alpha \cdot x$. The MAC key is (additively) secret-shared amongst all parties. Below, we show how parties use the MAC to generate *authenticated* secret shares of their input.

Implementing $\mathcal{F}_{\text{AuSS}}$ using SPDZ-like authenticated secret shares. Throughout this work, we construct our shuffle functionalities using $\mathcal{F}_{\text{AuSS}}$; however, for clarity and concreteness, we describe how the authenticated secret sharing scheme in SPDZ can implement the sub-functionalities/commands as defined in $\mathcal{F}_{\text{AuSS}}$.

Assuming 2 parties, let $\llbracket x \rrbracket := (\llbracket x \rrbracket_1, \llbracket x \rrbracket_2)$ denote the additive secret sharing of x , i.e. $\llbracket x \rrbracket_1 + \llbracket x \rrbracket_2 = x$; and similarly for the MAC key, $\llbracket \alpha \rrbracket$. A (2-party) “SPDZ-like” AuSS of x is a tuple of (1) an (additive) secret sharing of x , and (2) a sharing of its MAC $\gamma(x)$,

$$\langle x \rangle := (\llbracket x \rrbracket, \llbracket \gamma(x) \rrbracket)$$

One can verify that the sharing is linear where addition (+) and scalar multiplication ($\cdot$) are defined in the obvious way. Thus, we can naturally implement **Add**, **Mult** and **Scale**. See [22] for more details.

Briefly, a party, say P_1 , generates $\langle x \rangle$ for its input x using $\mathcal{F}_{\text{PREP}}$ wherein, it samples a uniformly random $r \xleftarrow{\$} \mathbb{F}$ and outputs $\langle r \rangle = (\llbracket r \rrbracket, \llbracket \gamma(r) \rrbracket) = \llbracket \alpha \cdot r \rrbracket$. In the online phase, $\llbracket r \rrbracket$ (i.e., only the value and not its MAC) is opened to P_1 . Subsequently, it broadcasts $d := x - r$. Both P_1 and P_2 update their share of r to *transform* it into a share of x as follows,

$$\begin{aligned} \llbracket x \rrbracket_1 &:= \llbracket r \rrbracket_1 \text{ and } \llbracket x \rrbracket_2 := \llbracket r \rrbracket_2 + d \\ \llbracket \gamma(x) \rrbracket_i &:= \llbracket \gamma(r) \rrbracket_i + \llbracket \alpha \rrbracket_i d \end{aligned}$$

Observe that $\llbracket x \rrbracket_1 + \llbracket x \rrbracket_2 = \llbracket r \rrbracket_1 + \llbracket r \rrbracket_2 + d = r + (x - r) = x$, and $\llbracket \gamma(x) \rrbracket_1 + \llbracket \gamma(x) \rrbracket_2 = \llbracket \gamma(r) \rrbracket_2 + \llbracket \alpha \rrbracket_1 d + \llbracket \gamma(r) \rrbracket_2 + \llbracket \alpha \rrbracket_2 d = \gamma(r) + \alpha d = \alpha(r + d) = \alpha x$. Thus, we obtain a correct AuSS of x . This is effectively the implementation of **Create**.

A SPDZ AuSS can implement **Open** by the parties simply revealing the first component of the secret share $\llbracket x \rrbracket$; i.e., they send their shares $\llbracket x \rrbracket_i$ to each other.

We implement **Check** using SPDZ AuSS as follows. First, using $\mathcal{F}_{\text{PREP}}$, we sample an authenticated secret share of a random value $\langle r' \rangle$ and reveal $\llbracket x \rrbracket + \llbracket r' \rrbracket = \llbracket x + r' \rrbracket$. Then, the parties compute $\llbracket s \rrbracket \leftarrow (\llbracket \gamma(x) \rrbracket + \llbracket \gamma(r') \rrbracket) - (x + r') \llbracket \alpha \rrbracket$. The parties now open s and check if it's equal to 0. If so, authentication passes and otherwise fails. This approach can be easily extended to succinctly check multiple authenticated secret shares at once using a random affine transformation. We refer the reader to [61, Fig. 2] for more details on how to implement **Check**.

SPDZ AuSS can be easily re-randomized, i.e. implement **ReRand**, by simply adding a secret-sharing of 0 to both shares in the tuple. For example, in the 2-party case, P_1 's new AuSS share for x is $\langle x \rangle_1 - (r_1, r_2)$ and P_2 's new share is $\langle x \rangle_2 + (r_1, r_2)$ for $r_1, r_2 \xleftarrow{\$} \mathbb{F}$. It is easy to check that the newly constructed AuSS is an authenticated secret share of the same value x . To prevent an incorrect sharing from being added (i.e., a sharing of a non-zero value), it is crucial for the parties to rerun the implementation above of **Check** on the newly constructed shares. We implicitly assume this is rerun such that **ReRand** is correctly implemented and outputs valid authenticated shares. Note that the re-randomization operation here is indeed “locally” linear in the input and randomness. Specifically, we mean there exists linear functions Π_{ReRand}^i such that P_i 's re-randomized share is equal to $\Pi_{\text{ReRand}}^i(\langle x \rangle_i, (r_1, r_2))$.

Finally, SPDZ AuSS can trivially implement **ReShare** by having parties *not in* S reshare their shares to the parties *in* S . This effectively sets their own share to

the zero tuple. For example, suppose we had a 3-party AuSS for parties P_1, P_2, P_3 of the value x that we'd like to reshare to $S = \{P_2, P_3\}$. We overload notation where its clear and now write $\llbracket x \rrbracket = (\llbracket x \rrbracket_1, \llbracket x \rrbracket_2, \llbracket x \rrbracket_3)$ and similarly for $\llbracket \gamma(x) \rrbracket$. Then, to **ReShare** $\langle x \rangle$ to S , P_1 randomly splits their shares $\llbracket x \rrbracket_1 = x_{1,2} + x_{1,3}$, and $\llbracket \gamma(x) \rrbracket_1 = x'_{1,2} + x'_{1,3}$ and sends $x_{1,i}, x'_{1,i}$ to party P_i that adds the values to its respective shares. We again implicitly assume that the parties run the implementation of **Check** above to ensure the output are valid authenticated secret shares. It is easy to verify that the newly constructed sharing only between P_2 and P_3 is a 2-party AuSS of the same underlying value x .

3.2 Linearly Homomorphic Encryption

We informally describe Linearly Homomorphic Encryption (LHE) below. We defer to [13] for a more formal definition.

A linearly homomorphic encryption scheme $\text{LHE} = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ for message space $\mathcal{M}$ and ciphertext space $\mathcal{C}$ consists of four efficient algorithms:

- $\text{KeyGen}(1^\lambda) \xrightarrow{\$} (pk, sk)$: On input the security parameter λ , output a public/secret key pair.
- $\text{Enc}(pk, m) \xrightarrow{\$} c$: On input pk and a message $m \in \mathcal{M}$, output a ciphertext $c \in \mathcal{C}$.
- $\text{Dec}(sk, c) \rightarrow m$: On input sk and ciphertext c , output the message m .
- $\text{Eval}(pk, \alpha, c) \rightarrow c$: On input pk , scalars $\alpha = (\alpha_1, \dots, \alpha_t) \in \mathbb{Z}$ and ciphertexts $c = (c_1, \dots, c_t)$ for any $t \geq 1$, output a ciphertext c .

For any two scalars α_1, α_2 and ciphertexts c_1, c_2 with respect to pk , we use operators “ $\cdot$ ” and “ $\odot$ ” as follows,

$$\alpha_1 \cdot c_1 \odot \alpha_2 \cdot c_2 = \text{Eval}(pk, (\alpha_1, \alpha_2), (c_1, c_2))$$

The scheme must satisfy the following:

Correctness. For any key pair $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$, messages $m_1, \dots, m_t \in \mathcal{M}$, scalars $\alpha_1, \dots, \alpha_t$, and ciphertexts $c_i \leftarrow \text{Enc}(pk, m_i)$,

$$\text{Dec}(sk, \text{Eval}(pk, \alpha, (c_1, \dots, c_t))) = \sum_{i=1}^t \alpha_i m_i.$$

Security. LHE satisfies semantic security (IND-CPA), i.e. no PPT adversary can distinguish encryptions of chosen messages.

Additionally, we require our LHE scheme to satisfy re-randomization and efficient ciphertext validation, which we formally define below.

Re-randomization. We use the definition from [42]. We explicitly show the randomness used by the encryption algorithm **Enc** as input. LHE supports

re-randomization if there exists a PPT algorithm ReRand such that the following distributions are statistically indistinguishable,

$$\left\{ \text{Enc}(pk, m, r_0) \mid r_0 \xleftarrow{\$} \text{coins}(\text{Enc}) \right\}, \text{ and} \\ \left\{ \text{ReRand}(\text{Enc}(pk, m, r_1), r) \mid r \xleftarrow{\$} \text{coins}(\text{ReRand}) \right\}$$

for all public keys pk , messages m and randomness r_1 . We implicitly run ReRand after every homomorphic ciphertext evaluation using Eval such that ciphertexts are *unlinkable*.

Efficient Ciphertext Validation. Observe that the indistinguishability condition under the definition of re-randomization only holds for valid ciphertexts, i.e., ciphertexts output by Enc . Thus, it leaves open the possibility of malformed ciphertexts in $\mathcal{D}$ to be *linkable* even after being “re-randomized” by ReRand . To prevent malformed ciphertexts, we require an efficient ciphertext validation algorithm.⁷

We say LHE supports efficient ciphertext validation if there exists an efficient PPT algorithm isValid such that for any m

$$\Pr \left[\text{isValid}(pk, c) = 1 \mid \begin{array}{l} (pk, sk) \leftarrow \text{KeyGen}(1^\lambda) \\ c = \text{Enc}(pk, m) \end{array} \right] = 1,$$

and for all PPT adversaries $\mathcal{A}$ and ciphertext $c \leftarrow \mathcal{A}(pk)$

$$\Pr \left[\text{isValid}(pk, c) = 1 \mid \forall m \in \mathcal{M}, r \in \text{coins}(\text{Enc}) : c \neq \text{Enc}(pk, m, r) \right] \leq \text{negl}(\lambda).$$

Furthermore, we also require the linear targeted malleability assumption or our “weakly predictable” assumption to hold for the LHE scheme.

As a simple example of a scheme that satisfies our set of conditions, we note that Paillier encryption satisfies re-randomization, efficient ciphertext validation, and was cited as a good candidate to satisfy linear targeted malleability in [8]. The ReRand algorithm is addition by an encryption of zero, which gives a uniformly random sample from ciphertexts with the same plaintext. The ciphertext validation check is the gcd algorithm, since the ciphertext space is the group of units $\mathbb{Z}_{n^2}^*$.

We note that our protocols work over any ring in which they are only polynomially-many zero divisors such that random linear combinations of non-zero values are not 0 except with negligible probability. We refer to [8, 11, 12, 32] and references therein for other “linear-only” and linear targeted malleability candidates constructed over rings. For simplicity, we demonstrate our results over a large finite field $\mathbb{F}$.

⁷ As presented, isValid is a non-interactive algorithm; however, it is possible to weaken our assumption on the LHE scheme to only require an interactive ciphertext validation protocol.

3.3 Malicious Security in the Ideal-Real World Paradigm

We follow the standalone simulator-based definition for (malicious) security with abort. For completeness, we outline the relevant definitions below. These are taken nearly verbatim from [53].

Definition 1 (Ideal World). Let $f: \{0,1\}^* \times \{0,1\}^* \rightarrow \{0,1\}^* \times \{0,1\}^*$ be a two-party functionality. Let $\mathcal{A}$ be a non-uniform probabilistic polynomial-time (PPT) machine and let $i \in \{1,2\}$ be the index of the corrupted party. Let $\mathcal{F}$ be the ideal functionality that implements f with abort (c.f. [53]). The ideal execution of f on inputs (x,y) , auxiliary input z to $\mathcal{A}$ and security parameter λ is defined as the output pair of the honest party and $\mathcal{A}$ from the execution of $\mathcal{F}$. Denote it by $\text{Ideal}_{f,\mathcal{A}(z),i}(x,y,\lambda)$.

Definition 2 (Real World). Let f be as above, and let π be a protocol that correctly computes f . Let $\mathcal{A}$ be a non-uniform PPT machine and let $i \in \{1,2\}$ be the index of the corrupted party. The real execution of π on inputs (x,y) , auxiliary input z to $\mathcal{A}$ and security parameter λ is defined as the output pair of the honest party and $\mathcal{A}$ from the execution of π . Denote it by $\text{Real}_{\pi,\mathcal{A}(z),i}(x,y,\lambda)$.

Definition 3 (Security with abort). Let f be a two-party functionality and let π be a protocol that computes f . Protocol π securely computes f with abort in the presence of static malicious adversaries if for every non-uniform PPT machine $\mathcal{A}$ in the real execution, there exists a non-uniform PPT machine $\mathcal{S}$ in the ideal execution such that for every $i \in \{1,2\}$,

$$\{\text{Ideal}_{f,\mathcal{S}(z),i}(x,y,\lambda)\}_{x,y,z,\lambda} \equiv_c \{\text{Real}_{f,\mathcal{A}(z),i}(x,y,\lambda)\}_{x,y,z,\lambda}$$

where $x,y,z \in \{0,1\}^*$, $|x| = |y|$, $\lambda \in \mathbb{N}$, and $\equiv_c$ is the standard notion of computational indistinguishability with respect to security parameter λ .

The definitions above can be generalized in the natural way to three parties in the honest majority setting since there is at most 1 corrupted party.

Throughout the paper, we drop writing the auxiliary input z , security parameter λ and inputs when it is clear.

We sequentially compose secure protocols for which we appeal to the *hybrid* model for security. See [53] for a detailed explanation.

4 Warm up: Three Party (Honest Majority) Shuffle

We begin by formally defining the ideal functionality for a half-shuffle, in Figure 2. Our definition covers both the three party honest majority case, where π is known to P_2 and P_3 , and the two party case, where π is known to P_2 . Informally, our definition states that the half-shuffle begins and ends with authenticated secret shares, and so the parties do not have any outputs other than `continue` or `abort`. To emulate this functionality in our simulation proofs, we must show that the abort probabilities are indistinguishable in the real and ideal world, and that

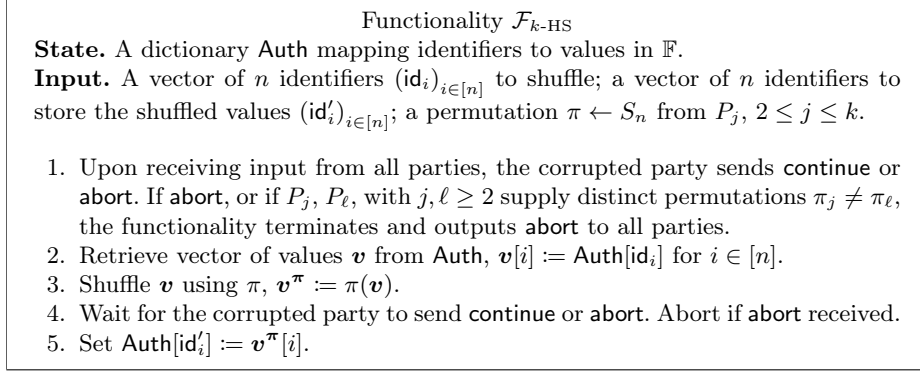


Fig. 2. $\mathcal{F}_{k\text{-HS}}$: Ideal Functionality for k -party half shuffle over authenticated shares for $k \in \{2, 3\}$

the probability that parties hold valid authenticated secret shares of $\pi(\mathbf{v})$ in the real world matches the probability that the corrupted party sends **continue** to the simulator in the ideal world.

We present the 3-party honest majority secure half-shuffle, $\Pi_{3\text{-HS}}$, in Protocol 2.

Correctness. Correctness follows by inspection and by definition of ReShare. P_1 performs a two party re-sharing of their shares in Step 1. P_2 and P_3 perform the desired permutation, followed by re-sharing the resulting shares with P_1 Step 2.

Efficiency. When each call to $\mathcal{F}_{\text{AuSS}}$ requires $O(1)$ computation and communication, the computation and communication cost of $\Pi_{3\text{-HS}}$ will be $O(n)$, by inspection. Any implicit calls to **Check** by ReShare can be performed in linear time (and indeed, when instantiating $\mathcal{F}_{\text{AuSS}}$ with SPDZ, with $O(1)$ communication, by batching).

Security. We prove security of $\Pi_{3\text{-HS}}$ using the security with abort model in the Ideal-Real world paradigm (c.f. Section 3.3), constructing a simulator with respect to each corrupted party. Security follows almost directly from the security of $\mathcal{F}_{\text{AuSS}}$ and thus, we defer the proofs to Appendix A.

5 Two-Party Maliciously Secure Shuffle

We present the two-party shuffle in $\Pi_{2\text{-HS}}$ (Protocol 3) in the $\{\mathcal{F}_{\text{PREP}}, \mathcal{F}_{\text{AuSS}}, \mathcal{F}_{\text{set-eq}}\}$ -*hybrid* model. As mentioned, we emulate the third party in the three-party hon-

Protocol 2 $\Pi_{3\text{-HS}}$: 3-Party Maliciously Secure Half-Shuffle

Input: Authenticated secret shares of vector, $\mathbf{v} \in \mathbb{F}^n$, $\langle \mathbf{v} \rangle$; P_2, P_3 with permutation, $\pi \leftarrow S_n$.

Output: $\langle \pi(\mathbf{v}) \rangle$, authenticated secret shares of permuted vector $\pi(\mathbf{v})$.

- 1: The parties re-share their shares using their shares **ReShare** on $\langle \mathbf{v} \rangle$ to $S = \{P_2, P_3\}$. Denote the re-shared AuSS as $\langle \mathbf{v} \rangle' =: \langle \mathbf{v} \rangle'_2 + \langle \mathbf{v} \rangle'_3$.
 - 2: P_2 and P_3 shuffles the shares and invokes **ReShare** on the shuffled shares with $S = \{P_1, P_2, P_3\}$ to obtain reshared AuSS as $\langle \mathbf{v}^\pi \rangle =: \langle \mathbf{v}^\pi \rangle_1 + \langle \mathbf{v}^\pi \rangle_2 + \langle \mathbf{v}^\pi \rangle_3$.
 - 3: Output $\langle \mathbf{v}^\pi \rangle$.
-

est majority shuffle using linearly homomorphic encryption. In particular, we *encrypt* to authenticated secret shares and perform homomorphic operations. Due to this, we cannot immediately use $\mathcal{F}_{\text{AuSS}}$ in a generic way. Instead, we require that any instantiation of $\mathcal{F}_{\text{AuSS}}$ implements **ReRand** as a *linear* operation over the input shares and the randomness.⁸ This enables re-randomization to occur on encrypted shares. Formally, we assume there exists two linear functions Π_{ReRand}^i for $i = \{1, 2\}$ such that for any two-party AuSS $\langle x \rangle$, $\Pi_{\text{ReRand}}^i(\langle x \rangle_i, r) = \langle x' \rangle_i$ where $\langle x' \rangle$ is the re-randomized share and r is the randomness fixed for $i = \{1, 2\}$.

Correctness. We verify correctness of our protocol via inspection. It is easy to see that when both parties are honest, $\Pi_{2\text{-HS}}$ outputs authenticated secret shares of $\pi(\mathbf{v})$.

Efficiency. It is easy to verify that the computation cost of $\Pi_{2\text{-HS}}$ is $O(n)$ when the cost of $\mathcal{F}_{\text{AuSS}}$ is $O(1)$ per operation, as in SPDZ. We require $O(n)$ secure multiplications in Step 12 and 16. When using SPDZ preprocessing for the $\mathcal{F}_{\text{PREP}}$ functionality, we can generate the necessary Beaver triples in $O(n)$ time. Moreover, communication is also $O(n)$ assuming constant ciphertext expansion of the LHE scheme.

Security. We prove security of $\Pi_{2\text{-HS}}$ using the security with abort model in the Ideal-Real world paradigm (c.f. Section 3.3).

Lemma 1. *Let $\text{LHE} = (\text{Keygen}, \text{Enc}, \text{Dec}, \text{Eval})$ be a re-randomizable homomorphic encryption scheme with efficient ciphertext validity checks. $\Pi_{2\text{-HS}}$ (Protocol 3) is secure against malicious P_1 in the $\{\mathcal{F}_{\text{PREP}}, \mathcal{F}_{\text{AuSS}}, \mathcal{F}_{\text{set-eq}}\}$ -hybrid model.*

Proof. Suppose the adversary $\mathcal{A}$ corrupts P_1 . We construct a simulator $\mathcal{S}_{2\text{-HS}}^1$ that interacts with $\mathcal{A}$ and $\mathcal{F}_{2\text{-HS}}$ to produce a transcript that is (computationally) indistinguishable from the real execution with $\mathcal{A}$. We assume that $\mathcal{S}_{2\text{-HS}}^1$ knows $\mathcal{A}$'s shares by simply simulating $\mathcal{F}_{\text{PREP}}$ and inspecting $\mathcal{A}$'s **Create** queries. We

⁸ To our knowledge, every known linear authenticated secret sharing scheme supports linear re-randomization. For example, the SPDZ AuSS (see Section 3.1).

Protocol 3 $\Pi_{2\text{-HS}}$: 2-Party Maliciously Secure Half-Shuffle

Input: $\langle v \rangle$, authenticated secret shares of $v \in \mathbb{F}^n$; P_2 with permutation, $\pi \leftarrow S_n$.

Output: $\langle \pi(v) \rangle$, authenticated secret shares of $\pi(v)$.

Part 1: commit to permutation

- 1: Use $\mathcal{F}_{\text{PREP}}$ to generate $\langle z \rangle$ for $z \xleftarrow{\$} \mathbb{F}$. Open $\llbracket z \rrbracket$ to P_2 .
- 2: P_2 computes $z^\pi := \pi(z)$ and AuSS $\langle z^\pi \rangle$ using Create.

Part 2: split and encrypt shares

- 3: P_1 generates two LHE key-pairs using independent, fresh randomness, $(pk_1, sk_1), (pk_2, sk_2) \leftarrow \text{LHE.KeyGen}(1^\kappa)$ and publishes pk_1 and pk_2 .
- 4: Use $\mathcal{F}_{\text{PREP}}$ to generate $\langle m \rangle$ for $m \xleftarrow{\$} \mathbb{F}^n$.
- 5: Compute $\langle a \rangle \leftarrow \langle v \rangle + \langle m \rangle$.
- 6: P_1 encrypts its shares,

$$C_a \leftarrow \text{LHE.Enc}(pk_1, \langle a \rangle_1), \text{ and } C_m \leftarrow \text{LHE.Enc}(pk_2, \langle m \rangle_1)$$

and sends C_a, C_m to P_2 .

- 7: P_2 checks the validity of C_a and C_m : $\text{isValid}(pk_1, C_a) \stackrel{?}{=} 1$ and $\text{isValid}(pk_2, C_m) \stackrel{?}{=} 1$. Abort if any invalid.
- 8: Use $\mathcal{F}_{\text{PREP}}$ to generate $\langle q_1 \rangle, \langle q_2 \rangle$ for $q_1, q_2 \xleftarrow{\$} \mathbb{F}^n$. Open $\langle q_1 \rangle, \langle q_2 \rangle$ to P_2 .
- 9: P_2 homomorphically computes, $C_{w,1} \leftarrow \sum_{i=1}^n q_1[i] \cdot C_a[i]$, and $C_{w,2} \leftarrow \sum_{i=1}^n q_2[i] \cdot C_m[i]$ and sends $C_{w,1}, C_{w,2}$ to P_1 .
- 10: P_1 decrypts $C_{w,1}, C_{w,2}$ and sets, $\langle w_i \rangle_1 := \text{LHE.Dec}(pk_i, C_{w,i})$ for $i = 1, 2$.
- 11: P_2 sets, $\langle w_1 \rangle_2 := \sum_{i=1}^n q_1[i] \cdot \langle a[i] \rangle_2$, $\langle w_2 \rangle_2 := \sum_{i=1}^n q_2[i] \cdot \langle m[i] \rangle_2$.
- 12: Compute, $\langle p_1 \rangle \leftarrow \langle w_1 \rangle - \sum_{i=1}^n \langle q_1[i] \rangle \cdot \langle a[i] \rangle$, and $\langle p_2 \rangle \leftarrow \langle w_2 \rangle - \sum_{i=1}^n \langle q_2[i] \rangle \cdot \langle m[i] \rangle$. Open $\langle p_1 \rangle$ and $\langle p_2 \rangle$. Abort if any value is non-zero.

Part 3: permute and rerandomize encrypted shares

- 13: P_2 samples vectors of randomness $r_1, r_2 \xleftarrow{\$} \text{coins}(\text{ReRand})$ and homomorphically computes,

$$C_a^\pi \leftarrow \Pi_{\text{ReRand}}^1(\pi(C_a), \text{LHE.Enc}(pk_1, r_1)) \text{ and } C_m^\pi \leftarrow \Pi_{\text{ReRand}}^1(\pi(C_m), \text{LHE.Enc}(pk_2, r_2))$$

Send C_a^π, C_m^π to P_1 .

- 14: P_2 sets $\langle a^\pi \rangle_2 \leftarrow \Pi_{\text{ReRand}}^2(\pi(\langle a \rangle_2), r_1)$, and $\langle m^\pi \rangle_2 \leftarrow \Pi_{\text{ReRand}}^2(\pi(\langle m \rangle_2), r_2)$.
- 15: P_1 decrypts and sets shares, $\langle a^\pi \rangle_1 \leftarrow \text{LHE.Dec}(sk_1, C_a^\pi)$, and $\langle m^\pi \rangle_1 \leftarrow \text{LHE.Dec}(sk_2, C_m^\pi)$.

Part 4: perform consistency checks

- 16: Jointly generate random nonzero weights $c_1, c_2, c_3 \xleftarrow{\$} \mathbb{F}$ from $\mathcal{F}_{\text{PREP}}$ (e.g. by partially opening random shares) and execute the four set equality checks:

$$\begin{aligned} (1) \quad \langle b_1 \rangle &\leftarrow \mathcal{F}_{\text{set-eq}}(\langle a \rangle, \langle a^\pi \rangle), & (2) \quad \langle b_2 \rangle &\leftarrow \mathcal{F}_{\text{set-eq}}(\langle m \rangle, \langle m^\pi \rangle), \\ (3) \quad \langle b_3 \rangle &\leftarrow \mathcal{F}_{\text{set-eq}}(\langle z \rangle, \langle z^\pi \rangle), & (4) \quad \langle b_4 \rangle &\leftarrow \mathcal{F}_{\text{set-eq}}(\langle c_1 a + c_2 m + c_3 z \rangle, \langle c_1 a^\pi + c_2 m^\pi + c_3 z^\pi \rangle) \end{aligned}$$

Execute $\mathcal{F}_{\text{AuSS.Check}}$ on each output. (These can be batched as in [22].) Abort if check fails. Partially open $\llbracket b_i \rrbracket$. Abort if any non-zero.

- 17: Parties create shares of the permuted vector by removing shares of mask, $\langle v^\pi \rangle \leftarrow \langle a^\pi \rangle - \langle m^\pi \rangle$
 - 18: Return $\langle v^\pi \rangle$.
-

may list the corresponding step in Protocol 3 in parenthesis beside each step of the simulator for clarity. $\mathcal{S}_{\text{HS}}^1$ aborts at any point $\mathcal{A}$ aborts.

$\mathcal{S}_{2\text{-HS}}^1$ runs as follows.

1. Sample $\pi' \xleftarrow{\$} S_n, \mathbf{z}' \xleftarrow{\$} \mathbb{F}^n$. Use $\mathcal{F}_{\text{PREP}}$ to generate $\langle \pi'(\mathbf{z}') \rangle$.
2. Obtain pk_1, pk_2 from $\mathcal{A}$ (Step 3).
3. Obtain $C_{\mathbf{a}}^*, C_{\mathbf{m}}^*$ from $\mathcal{A}$ (Step 6).
4. Check their validity just like in Step 7. Abort if any invalid.
5. Generate authenticated shares $\langle \mathbf{q}_1 \rangle, \langle \mathbf{q}_2 \rangle$ for $\mathbf{q}_1, \mathbf{q}_2 \xleftarrow{\$} \mathbb{F}^n$ using $\mathcal{F}_{\text{PREP}}$.
6. Compute $\sum_{i=1}^n \mathbf{q}_1[i] \cdot C_{\mathbf{a}}^*[i]$ and $\sum_{i=1}^n \mathbf{q}_2[i] \cdot C_{\mathbf{m}}^*[i]$ to obtain $C_{w,1}^*$ and $C_{w,2}^*$ respectively. Send $C_{w,1}^*$ and $C_{w,2}^*$ to $\mathcal{A}$.
7. If $\mathcal{A}$ uses the wrong shares⁹ (for $w_i, \mathbf{a}$ or $\mathbf{m}$), abort and send **abort** to $\mathcal{F}_{2\text{-HS}}$ (Step 12). Else partially open to value 0 using $\mathcal{F}_{\text{PREP}}$.
8. Sample $\mathbf{r}_1^*, \mathbf{r}_2^*$ uniformly at random from the share space, and encrypt,

$$C_{\mathbf{a}}^{\pi,*} \leftarrow \text{LHE.Enc}(pk_1, \mathbf{r}_1^*), \text{ and } C_{\mathbf{m}}^{\pi,*} \leftarrow \text{LHE.Enc}(pk_2, \mathbf{r}_2^*)$$

Send $C_{\mathbf{a}}^{\pi,*}$ and $C_{\mathbf{m}}^{\pi,*}$ to $\mathcal{A}$.

9. If $\mathcal{A}$ uses the wrong inputs (not equal to $\mathbf{r}_1^*, \mathbf{r}_2^*$) for any of the set equality checks, abort and send **abort** to $\mathcal{F}_{2\text{-HS}}$ (Step 16). Else partially open to $b_i = 0$ for all $i = 1, \dots, 4$.
10. Invoke $\mathcal{F}_{2\text{-HS}}$ with input share $\langle \mathbf{v} \rangle_1$ and receive share $\langle \mathbf{v}^\pi \rangle_1$.
11. When $\mathcal{A}$ sends query to subtract $\langle \mathbf{m}^\pi \rangle$ from $\langle \mathbf{a}^\pi \rangle$ (implemented using **Add**), respond with share $\langle \mathbf{v}^\pi \rangle_1$.
12. Output whatever $\mathcal{A}$ outputs.

Security follows almost immediately due to the security of $\mathcal{F}_{\text{AuSS}}$ and $\mathcal{F}_{\text{PREP}}$, and the fact that $\mathcal{S}_{2\text{-HS}}^1$ behaves nearly identically to P_2 in $\Pi_{2\text{-HS}}$. Formally, we prove security via a series of hybrids between the ideal and real execution. Denote by $f_{2\text{-HS}}$ the two-party shuffling functionality which takes as input id's of values to be shuffled (and which id's to store the shuffled values) and a permutation π from P_2 .

- **H₀**: $\mathcal{A}$'s transcript when interacting with $\mathcal{S}_{2\text{-HS}}^1$. This is the ideal-world execution, $\text{Ideal}_{f_{2\text{-HS}}, \mathcal{S}_{2\text{-HS}}^1, 1}$.
- **H₁**: Replace π' in Step 1 of $\mathcal{S}_{2\text{-HS}}^1$ with P_2 's actual input in $\Pi_{2\text{-HS}}$, π .
- **H₂**: Replace Steps 4–7 with $\Pi_{2\text{-HS}}$ Steps 7–12.
- **H₃**: Replace $C_{\mathbf{a}}^{\pi,*}, C_{\mathbf{m}}^{\pi,*}$ with $C_{\mathbf{a}}^\pi, C_{\mathbf{m}}^\pi$ as in Π_{HS} .
- **H₄**: Replace Step 9 with Step 16 of $\Pi_{2\text{-HS}}$.
- **H₅**: Replace Step 11 with Step 17 of $\Pi_{2\text{-HS}}$. This is the real execution, $\text{Real}_{\Pi_{2\text{-HS}}, \mathcal{A}, 1}$.

⁹ We remark that “wrong” is well-defined as $\mathcal{S}_{2\text{-HS}}^1$ knows $\mathcal{A}$'s shares of $\mathbf{a}$ and $\mathbf{m}$, and it expects $\langle w_1 \rangle_1 = \sum \mathbf{q}_1[i] \cdot \langle \mathbf{a}[i] \rangle_1$ (and similarly for $\langle w_2 \rangle_1$).

$\mathbf{H}_1$ is indistinguishable from $\mathbf{H}_0$ by the security of $\mathcal{F}_{\text{AuSS}}$. Between $\mathbf{H}_2$ and $\mathbf{H}_1$, we again leverage $\mathcal{F}_{\text{AuSS}}$'s security for $\mathbf{q}_1$ and $\mathbf{q}_2$. Observe that $\mathcal{S}_{\text{HS}}^1$'s steps are identical to P_2 's up until the abort condition. We show that the difference in the probability of abort is negligible. WLOG, we focus on $\mathbf{a}$. Suppose $\mathcal{A}$ uses the wrong shares $\mathbf{a}_1^*$ to construct $\mathbf{C}_a^*$. Then the check in step 12 will only succeed if $\mathcal{A}$ supplies some $\langle w_1 \rangle_1^*$ such that $\langle w_1 \rangle_1^* = \sum \mathbf{q}_1[i] \cdot \mathbf{a}[i] - \langle w_1 \rangle_2$, or equivalently, since $\mathcal{A}$ learns $\langle w_1 \rangle_1 = \text{LHE.Dec}(pk_i, C_{w,1}) = \sum \langle \mathbf{a}^*[i] \rangle_1 \mathbf{q}_1[i]$, $\mathcal{A}$ must be able to guess some value $\langle w_1 \rangle_1^*$ satisfying $\langle w_1 \rangle_1^* - \langle w_1 \rangle_1 = \sum_{i=1}^n \mathbf{q}_1[i] (\langle \mathbf{a}[i] \rangle_1 - \langle \mathbf{a}^*[i] \rangle_1)$, but this expression is uniformly random, by the randomness of the vector $\mathbf{q}_1$. Similarly if $\mathcal{A}$ uses the correct $\mathbf{C}_a, \mathbf{C}_m$, but alters their shares $\langle w_1 \rangle_1^*$, they will guarantee the check in Step 12 fails, by the security of $\mathcal{F}_{\text{AuSS}}$, since the check will pass with the correct shares. Therefore $\mathcal{S}_{\text{HS}}^1$ aborts implies P_2 aborts in $\Pi_{2\text{-HS}}$ with probability at least $1 - 1/|\mathbb{F}|$. Conversely, suppose the opening of p_1 is not equal to 0. By correctness of $\Pi_{2\text{-HS}}$, this implies $\mathcal{A}$ used the wrong shares. Therefore, $\mathbf{H}_2 \equiv_c \mathbf{H}_1$. $\mathbf{H}_3$ is computationally indistinguishable from $\mathbf{H}_2$ as the underlying plaintexts are uniformly random in both hybrids, the ciphertexts are valid, and LHE is re-randomizable. Next, due to the security of $\mathcal{F}_{\text{AuSS}}$, if $\mathcal{A}$ uses the wrong shares, then Check aborts. Therefore, the difference in abort probability between $\mathbf{H}_4$ and $\mathbf{H}_3$ is negligible. Lastly, $\mathbf{H}_5 \equiv_c \mathbf{H}_4$ since by the security of $\mathcal{F}_{\text{AuSS}}$ shares look effectively random subject to the constraint that the output is consistent with the shares of the honest party. That is, they must reconstruct to a valid permutation of the input. Indeed $\mathcal{S}_{\text{HS}}^1$ returns the consistent share to $\mathcal{A}$ as it's the output of $\Pi_{2\text{-HS}}$.

Since each (constant-number of) intermediate hybrids is indistinguishable, we conclude $\text{Ideal}_{f_{2\text{-HS}}, \mathcal{S}_{2\text{-HS}}^1, 1} = \mathbf{H}_0 \equiv_c \mathbf{H}_5 = \text{Real}_{\Pi_{2\text{-HS}}, \mathcal{A}, 1}$. Thus, $\Pi_{2\text{-HS}}$ is secure against malicious P_1 .

Next, we prove security with respect to malicious P_2 .

Lemma 2. *Let $\text{LHE} = (\text{Keygen}, \text{Enc}, \text{Dec}, \text{Eval})$ be a homomorphic encryption scheme with linear targeted malleability, in the sense of [8]. Then $\Pi_{2\text{-HS}}$ (Protocol 3) is secure against malicious P_2 in the $\{\mathcal{F}_{\text{PREP}}, \mathcal{F}_{\text{AuSS}}, \mathcal{F}_{\text{set-eq}}\}$ -hybrid model.*

Proof. Suppose the adversary $\mathcal{A}$ corrupts P_2 . We construct a simulator $\mathcal{S}_{2\text{-HS}}^2$ that interacts with $\mathcal{A}$ and $\mathcal{F}_{2\text{-HS}}$ to produce a transcript that is (computationally) indistinguishable from the real execution with $\mathcal{A}$. $\mathcal{S}_{2\text{-HS}}^2$ runs as follows.

1. Extract $\mathbf{z}$ from $\mathcal{F}_{\text{PREP}}$ and the possibly malicious input $\pi(\mathbf{z})^*$ from $\mathcal{F}_{\text{AuSS}}$. If $\mathbf{z}$ and $\pi(\mathbf{z})^*$ are identical as multisets, then $\mathcal{S}_{2\text{-HS}}^2$ defines the permutation π to be the lexicographically first permutation such that $\pi(\mathbf{z})^* = \pi(\mathbf{z})$. Otherwise, $\mathcal{S}_{2\text{-HS}}^2$ defines π to be the zero map and aborts during the set equality checks.
2. Generate the messages $\mathbf{C}_a, \mathbf{C}_m$ received in Step 6 as encryptions of uniformly random values under the correct public keys.
3. Extract the vectors $\mathbf{q}_1, \mathbf{q}_2$ generated in Step 8 from $\mathcal{F}_{\text{PREP}}$. Let $\mathbf{C}_{w,1}^*, \mathbf{C}_{w,2}^*$ be the messages sent by $\mathcal{A}$ during Step 8 of $\Pi_{2\text{-HS}}$. Invoke the simulator

from the linear targeted malleability to obtain the possibly malicious affine transformations

$$\mathbf{b}_1 \mapsto q_{0,1}^* + \sum_{i=1}^n \mathbf{q}_1^*[i] \mathbf{b}_1[i]$$

and

$$\mathbf{b}_2 \mapsto q_{0,2}^* + \sum_{i=1}^n \mathbf{q}_2^*[i] \mathbf{b}_2[i]$$

which, when acting on $\mathbf{a}, \mathbf{m}$ give outputs indistinguishable from the decryptions w_1^*, w_2^* . If $\mathbf{q}_1^* \neq \mathbf{q}_1$ or $\mathbf{q}_2^* \neq \mathbf{q}_2$, or if $\langle w_1 \rangle_2 \neq -q_{0,1}^* + \sum_{i=1}^n \mathbf{q}_1^*[i] \langle \mathbf{a}[i] \rangle_2$ or $\langle w_2 \rangle_2 \neq -q_{0,2}^* + \sum_{i=1}^n \mathbf{q}_2^*[i] \langle \mathbf{m}[i] \rangle_2$, generate a random nonzero opening value in Step 12 and abort; otherwise output zero in Step 12 and proceed.

4. $\mathcal{A}$ sends possibly malicious $\mathbf{C}_a^\pi, \mathbf{C}_m^\pi$ in Step 12. Extract from the simulator for linear targeted malleability the corresponding affine transformations, and if possible, compute from these affine transformations the pairs $(\mathcal{L}_1, \mathbf{r}_1^*)$ and $(\mathcal{L}_2, \mathbf{r}_2^*)$ such that the extracted transformations are:

$$\Pi_{\text{ReRand}}^1(\mathcal{L}_1(\cdot), \mathbf{r}_1^*) \quad \text{and} \quad \Pi_{\text{ReRand}}^1(\mathcal{L}_2(\cdot), \mathbf{r}_2^*).$$

If this is not possible, abort.

5. Compute the implied outputs $\langle \mathbf{a}^\pi \rangle_2 := \Pi_{\text{ReRand}}^2(\mathcal{L}_1(\langle \mathbf{a}_2 \rangle), \mathbf{r}_1^*)$ and $\langle \mathbf{a}^\pi \rangle_2 := \Pi_{\text{ReRand}}^2(\mathcal{L}_2(\langle \mathbf{a}_2 \rangle), \mathbf{r}_2^*)$ and obtain three random weights $c_1, c_2, c_3 \in \mathbb{F}$ from $\mathcal{F}_{\text{PREP}}$. Then, extract from $\mathcal{A}$ the inputs to the four set equality checks in Step 16, and abort if any of the inputs disagree with the computed shares, or if either of $\mathcal{L}_1, \mathcal{L}_2$ are not equal to the permutation matrix corresponding to π , or if π is the zero map.
6. If $\mathcal{A}$ does not abort, invoke $\mathcal{F}_{2\text{-HS}}$ with inputs $(\langle \mathbf{v} \rangle_2, \pi)$ and output whatever $\mathcal{A}$ outputs.

We prove computational indistinguishability from real-world execution with a series of hybrid models.

- $\mathbf{H}_0$: Ideal-world execution, $\text{Ideal}_{f_{2\text{-HS}}, \mathcal{S}_{2\text{-HS}}^2, 2}$.
- $\mathbf{H}_1$: Replace the randomly generated messages in Step 2 of $\mathcal{S}_{2\text{-HS}}^2$ with the real-world messages sent in Step 6 and the validity check in Step 7 of $\Pi_{2\text{-HS}}$.
- $\mathbf{H}_2$: Replace Step 3 of $\mathcal{S}_{2\text{-HS}}^2$ with Steps 8–12 of $\Pi_{2\text{-HS}}$.
- $\mathbf{H}_3$: Replace Steps 4–6 of the $\mathcal{S}_{2\text{-HS}}^2$ with Steps 12–18 of $\Pi_{2\text{-HS}}$. This is the real-world execution of the protocol, $\text{Real}_{\Pi_{2\text{-HS}}, \mathcal{A}, 1}$.

$\mathbf{H}_0$ and $\mathbf{H}_1$ are indistinguishable by the IND-CPA property of the homomorphic encryption system (even knowing $\mathbf{m}$ and $\mathbf{v}$, the adversary cannot distinguish $\text{Enc}(pk_1, \llbracket \mathbf{a} \rrbracket_1)$ and the encryption of a random value) and the definition of `isValid` as the simulator generates valid ciphertexts.

$\mathbf{H}_1$ and $\mathbf{H}_2$ are indistinguishable during an honest execution of the protocol, since the checks in Step 12 of Π_{HS} are guaranteed to pass. If the values $\mathbf{C}_{w,1}^*, \mathbf{C}_{w,2}$ are generated with maliciously chosen affine transformations such that

$\mathbf{q}_1^* \neq \mathbf{q}_1$ or $\mathbf{q}_2^* \neq \mathbf{q}_2$), then the first check in Step 12 can be transformed by $\mathcal{A}$ to give

$$q_{0,1}^* + \langle w_1 \rangle_2^* - \sum \mathbf{q}_1[i] \langle \mathbf{a}[i]^* \rangle_2 = \sum_{i=1}^n (q_1[i] - q_1[i]^*) \langle \mathbf{a}[i] \rangle,$$

with a similar expression for $\mathbf{q}_2$ and $\mathbf{m}$. Each term on the left-hand side is known to $\mathcal{A}$, but the vectors $\mathbf{a}, \mathbf{m}$ are random and not known to P_2 , so the j -th check fails with overwhelming probability if any of the $(q_j[i] - q_j[i]^*)$ terms are nonzero. Similarly if a nonzero value $q_{0,j}^*$ is used as part of the affine transformation in Step 8, and this term is not removed by P_2 's choice of $\langle w_j \rangle_2$ in Step 11, the Check in Step 11 will fail.

$\mathbf{H}_2$ and $\mathbf{H}_3$ are indistinguishable: The only messages P_2 receives during Steps 12 through 18 of the real world protocol are during the execution of the set equality check in Steps 15 and 16, so the views of these two hybrids will be indistinguishable as long as the abort probability of the set equality check in the $\mathbf{H}_2$ and $\mathbf{H}_3$ are indistinguishable. If the conditions the simulator checks in Step 5 of the simulated protocol all hold, then $\mathbf{H}_3$ could only abort if the entries of one of the vectors $\mathbf{a}, \mathbf{m}, \mathbf{z}$ are not unique, in which case $\mathcal{L}_1, \mathcal{L}_2$, or the π^* used by $\mathcal{A}$ might not match the π computed by $\mathcal{S}$, but the real world checks would still pass. The probability of this is negligible, since $\mathbf{a}, \mathbf{m}, \mathbf{z}$ are uniformly random, by the security of $\mathcal{F}_{\text{PREP}}$.

Otherwise, one of the conditions of the simulated Step 5 will fail. If $\mathcal{L}_1 \neq \pi$, then either $\langle \mathbf{a}^\pi \rangle_2 + \langle \mathbf{a}_1^\pi \rangle_1$ is not equal as a multiset to $\langle \mathbf{a} \rangle$, and so the first set equality check will fail, or it equals $\sigma(\langle \mathbf{a} \rangle)$, for some $\sigma \neq \pi$. If the fourth set equality check passes, there exists some permutation τ with

$$\langle c_1 \mathbf{a}^\pi + c_2 \mathbf{m}^\pi + c_3 \mathbf{z}^\pi \rangle = \tau(\langle c_1 \mathbf{a} + c_2 \mathbf{m} + c_3 \mathbf{z} \rangle).$$

Taking the i -th entry of this equality gives:

$$c_1 \mathbf{a}[\sigma^{-1}(i)] + c_2 \mathbf{m}^\pi[i] + c_3 \mathbf{z}[\pi^{-1}(i)] = c_1 \mathbf{a}[\tau^{-1}(i)] + c_2 \mathbf{m}[\tau^{-1}(i)] + c_3 \mathbf{z}[\tau^{-1}(i)].$$

There is some i for which $\sigma^{-1}(i) \neq \pi^{-1}(i)$, and so $\tau^{-1}(i)$ is different from at least one of these two values. By the randomness of c_1 and c_3 , this equality will only hold with negligible probability.

The argument if $\mathcal{L}_2 \neq \pi$ is identical, looking instead at $\langle \mathbf{m} \rangle$ and $\langle \mathbf{m}^\pi \rangle$. If π is the zero map in the ideal world, then $\mathbf{z} \neq \mathbf{z}^{\pi^*}$ as multisets in the real world, and the third set equality check will fail. Finally, if $\mathcal{L}_1, \mathcal{L}_2, \pi$ are all correct, but the wrong entries are used in the set equality check, $\mathcal{F}_{\text{AuSS}}$ will reject the inputs. Thus under any condition where $\mathcal{S}$ aborts during its Step 5, the real world protocol will abort during Step 16, except with negligible probability.

6 Weakly Predicable Encryption

In this section, we will give an alternative security notion, that of *weakly predicable* homomorphic encryption. The terminology here refers to the idea that an

adversary who is performing an attack via selective abort can only test weak predicates that reveal a small amount of information about the underlying data. Although, like the notions of “linear-only” encryption and linear targeted malleability, this is a non-falsifiable assumption, we show that it is a weaker assumption than “linear-only” encryption, and sufficient to prove the security of our two-party malicious shuffle protocol.

We begin with the following security definition.

Definition 4. A homomorphic encryption scheme $\text{HE} = (\text{Keygen}, \text{Enc}, \text{Dec}, \text{Eval})$, with $\text{Enc} : (pk, M, R) \rightarrow G$ is weakly predicable if for every polynomial time algorithm $T : pk \times G^k \times \{0, 1\}^{\text{poly}(\kappa)} \rightarrow G$ and every auxiliary input $z \in \{0, 1\}^{\text{poly}(\kappa)}$ there exists some polynomial-time computable set $S_{T, \mathbf{c}, z} \subseteq M^k$, such that either $|S_{T, \mathbf{c}, z}| < \text{negl}(\kappa)|M|^k$ and for any plaintext generator $\mathcal{M}(pk)$, with $\mathbf{c} = \text{Enc}(\mathbf{a} \leftarrow \mathcal{M}(pk))$, we have:

$$\Pr [\text{Dec}(sk, T(pk, \mathbf{c}, z)) = 0 \wedge \mathcal{M}(pk) \notin S_{T, \mathbf{c}, z}] \leq \text{negl}(\kappa),$$

or $S_{T, z} = M^k$ and

$$\Pr [\text{Dec}(sk, T(pk, \text{Enc}(\mathbf{c}), z)) = 0] \geq 1 - \text{negl}(\kappa)$$

Here the probability is over the plaintext generator $\mathcal{M}$. We can restrict our definition to consider “benign” auxiliary input distributions as in [8].

Remark 1. We make two observations about our definition. First, the definition can be understood as a generalization of the definition of linear-only homomorphic encryption, where instead of extracting the linear transformation performed homomorphically by the adversary, we extract the zero set of the (not necessarily linear) computation homomorphically performed by the adversary, and require this zero set or its complement to be small.

Second, the terminology of weak predicability comes from the predicate $\text{Dec}(sk, T(pk, \text{Enc}(\mathbf{c}), z)) = 0$, which corresponds to some predicate on the underlying plaintext $\mathbf{a}$. We consider this a *strong predicate* when the probability of it being satisfied and the probability of it being unsatisfied over a random choice of $\mathbf{a}$ are both positive constants as $\lambda \rightarrow \infty$, and a *weak predicate* if one of these probabilities is $\leq \text{negl}(\kappa)$. We desire our homomorphic encryption system to only support weak predicates, hence it is *weakly predicable*.

Lemma 3. Let $\text{LHE} = (\text{Keygen}, \text{Enc}, \text{Dec}, \text{Eval})$ be a linear-only homomorphic encryption scheme, in the sense of [8], with $|M| = 2^{\omega(\kappa)}$. Then, LHE is weakly predicable.

Proof. By the definition of linear-only homomorphic encryption, there is an extractor $E(pk, c_1, \dots, c_m; z)$ that outputs a linear transformation $(\Pi, \mathbf{b})$ representing the affine transformation the adversary performs on the ciphertexts. Then the set $S_{T, \mathbf{c}, z}$ can be computed as the zero set of this affine transformation, which is either a proper subspace, or the entire space. When the zero set is a proper subspace, its size is $\leq |M|^{n-1} = \text{negl}(\kappa)|M|$. Therefore linear-only homomorphic encryption is weakly predicable.

We note that our proof of Lemma 1 did not use the linear-only assumption anywhere, and so Protocol 3 remains secure against malicious P_1 under the weaker assumption that the homomorphic encryption is weakly predicable. We now show security against malicious P_2 .

Lemma 4. *Let $\text{LHE} = (\text{Keygen}, \text{Enc}, \text{Dec}, \text{Eval})$ be weakly predicable. Then Protocol 3 is secure against malicious P_2 .*

Proof. We construct a modified simulator $\mathcal{S}_{2\text{-HS}}^{2'}$, which behaves the same as the simulator in the proof of Lemma 2, with some modifications to Steps 3 through 5. On those steps, instead of invoking the polynomial extractor, the simulator instead uses the weakly predicable property to deduce the cheating probability of the adversary.

For a given adversary $\mathcal{A}$, we define the following polynomial time algorithms:

$$\begin{aligned} T_1(pk, \text{Enc}_1(\mathbf{a}), z) &:= \mathbf{C}_{w,1}^* - \sum_{i=1}^n \mathbf{q}_1[i] \cdot (\mathbf{C}_{\mathbf{a}}[i] - \langle \mathbf{a} \rangle_2[i]) - \langle w_1 \rangle_2 \\ T_2(pk, \text{Enc}_2(\mathbf{m}), z) &:= \mathbf{C}_{w,2}^* - \sum_{i=1}^n \mathbf{q}_2[i] \cdot (\mathbf{C}_{\mathbf{m}}[i] - \langle \mathbf{m} \rangle_2[i]) - \langle w_2 \rangle_2 \\ T_3(pk, \text{Enc}_1(\mathbf{a}), z) &:= \pi^{-1}(\mathbf{C}_{\mathbf{a}}^{\pi*} + \langle \mathbf{a}^{\pi} \rangle_2) - \langle \mathbf{a} \rangle_2 - \mathbf{C}_{\mathbf{a}} \\ T_4(pk, \text{Enc}_2(\mathbf{m}), z) &:= \pi^{-1}(\mathbf{C}_{\mathbf{m}}^{\pi*} + \langle \mathbf{m}^{\pi} \rangle_2) - \langle \mathbf{m} \rangle_2 - \mathbf{C}_{\mathbf{m}}, \end{aligned}$$

where the starred ciphertexts are supplied by the adversary during the execution of the protocol. Since the adversary can be executed in polynomial time with the auxiliary input z , there is a polynomial time to generate the (possibly malicious) ciphertexts $\mathbf{C}_{w,1}^*, \mathbf{C}_{w,2}^*, \mathbf{C}_{\mathbf{a}}^{\pi*}, \mathbf{C}_{\mathbf{m}}^{\pi*}$ and then the expressions above. Informally, these polynomial time algorithms represent the simulator interacting with the adversary to generate a ciphertext which should be an encryption of zero if the adversary has behaved honestly.

Let S_1, S_2, S_3, S_4 be the corresponding sets from the definition of weakly predicable encryption. The simulator $\mathcal{S}_{2\text{-HS}}^{2'}$ aborts in Step 3 if S_1 or S_2 is not equal to the entire space, and aborts in Step 5 if S_3 or S_4 is not equal to the entire space.

The proof of computational indistinguishability is also similar to the proof in Lemma 2. We write $\mathbf{H}'_0, \mathbf{H}'_1, \mathbf{H}'_2, \mathbf{H}'_3$ for hybrid models that are identical to the hybrids in Lemma 2, replacing $\mathcal{S}_{2\text{-HS}}^2$ with $\mathcal{S}_{2\text{-HS}}^{2'}$ everywhere it appears. We go through the modifications to the proofs of indistinguishability between $\mathbf{H}'_1$ and $\mathbf{H}'_2$ and between $\mathbf{H}'_2$ and $\mathbf{H}'_3$.

$\mathbf{H}'_1$ and $\mathbf{H}'_2$ are indistinguishable: If $S_1 = S_2 = M^n$, then the adversary behaves honestly in transforming the ciphertexts in Step 9 of the real world protocol, and uses shares $\langle w_j \rangle_2$ in Step 12 consistent with these ciphertexts, and so P_1 will not abort in Step 12.

If S_1 or S_2 is not equal to the entire space, then at least one of the expressions computed by T_1, T_2 is an encryption of a nonzero value with overwhelming probability over a random choice of $\mathbf{a}, \mathbf{m}$, which would imply the adversary's messages

in Step 11 are inconsistent with the ciphertexts sent in Step 8. However, we are still not quite done; since $S_1 := S_{T_1, \mathbf{c}, z}$ depends on $\mathbf{c}$, which in turn depends on $\mathbf{a}$, we cannot immediately conclude that $\Pr[\mathbf{a} \in S_1] \leq \text{negl}(\kappa)$. By way of contradiction, however, if this probability were non-negligible, we could use this to break the IND-CPA property of the cryptosystem. Indeed, an adversary could use two uniformly random vectors $\mathbf{a}, \mathbf{b}$ as its plaintexts, compute $S_{T_1, \mathbf{c}, z}$ using the challenge ciphertext $\mathbf{c}$, and, if exactly one of $\mathbf{a}, \mathbf{b}$ lies in $S_{T_1, \mathbf{c}, z}$, guess that vector as the plaintext. Otherwise, guess randomly. If $\Pr[\mathbf{a} \in S_{T_1, \text{Enc}(\mathbf{a}), z}]$ is non-negligible, while $\Pr[\mathbf{a} \in S_{T_1, \text{Enc}(\mathbf{b}), z}]$ is negligible, this gives the adversary a non-negligible advantage in the IND-CPA game.

The argument for the indistinguishability of $\mathbf{H}'_2$ and $\mathbf{H}'_3$ is similar. Again, if $S_3 = S_4 = M^n$, the real world abort probability is negligible, adding in, as in the proof for linear-only encryption, the negligible probability that $\mathbf{z}$ contains repeated elements. If S_3 or S_4 is a negligible fraction of M^n , then with overwhelming probability either T_3 or T_4 gives a nonzero output, and so the first or second set equality check in Step 16 of the real world protocol will fail.

7 Three-Party Shuffle Without Authenticated Re-sharing

In this section, we present a variant of the three-party protocol that converts from authenticated shares to simple additive secret shares during the shuffle step. We present this variant as an example of the range of choices available to the protocol designer when deciding what to authenticate and how.

In our main protocol, we carried along the authenticated shares with us everywhere. This limits the sorts of adversarial attacks that may occur, but it also requires, in the two-party case, the choice of an AuSS scheme and a homomorphic encryption scheme that are compatible. When using SPDZ for the AuSS scheme, this requires twice as many ciphertexts. The protocol variant we present here holds the potential to be concretely more efficient than the always-authenticate approach, depending on the schemes that are chosen.

The blueprint is as follows. The parties perform a bespoke resampling to distribute *unauthenticated* shares to P_2 and P_3 , who perform the shuffle and send unauthenticated shares back. After this, all three parties invoke `Create` from $\mathcal{F}_{\text{AuSS}}$ to authenticate their shares. We perform checks along the way to make sure that the stored values are correct and show that these checks are not vulnerable to selective failure attacks. We give the following protocol as Protocol 4 and proceed with the security proof.

Lemma 5. *Protocol 4 is secure against malicious P_1 .*

Proof. Informally, P_1 has two opportunities to cheat, in its call to `Create` in steps 4 and 6. Cheating in Step 4 will be detected by the affine transformation in Step 4, and cheating in step 6 will be detected by the subtraction of dot products in Step 8, which can be understood as a bespoke set equality check for three parties when two parties know the permutation. Formally, the simulator $\mathcal{S}$ interacts with the ideal functionality and the adversary $\mathcal{A}$ as follows:

1. $\mathcal{S}$ generates uniformly random messages w_2, w_3 , gives them to $\mathcal{A}$ and extracts $\mathcal{A}$'s subsequent input w^* to **Create**. If $w^* \neq w_2 + w_3$, $\mathcal{S}$ outputs $w^* - w_2 - w_3$ and aborts.
2. $\mathcal{S}$ generates a uniformly random message $\mathbf{v}_{2,1}$, invokes the ideal functionality to obtain $[\mathbf{v}^\pi]_1$, and subtracts to obtain $\mathbf{v}_{3,1} = [\mathbf{v}^\pi]_1 - \mathbf{v}_{2,1}$. $\mathcal{S}$ then extracts $\mathcal{A}$'s input $[\mathbf{v}^\pi]_1^*$ to **Create**, and aborts if $[\mathbf{v}^\pi]_1^* \neq [\mathbf{v}^\pi]_1$ or if $\mathcal{A}$ aborts, and otherwise outputs whatever $\mathcal{A}$ outputs.

We prove computational indistinguishability with a series of hybrids.

- $\mathbf{H}_0$: $\mathcal{A}$'s transcript when interacting with $\mathcal{S}$. This is the ideal-world execution, $\text{Ideal}_{f_{3\text{-HS}}, \mathcal{S}, 1}$.
- $\mathbf{H}_1$: Replace the simulated w_2 and w_3 with the real world messages, and execute the real-world checks in Step 4.
- $\mathbf{H}_2$: Replace the simulated $\mathbf{v}_{2,1}$ and $\mathbf{v}_{3,1}$ with the real world messages, and execute the real-world checks in Step 8. This is the real execution, $\text{Real}_{\Pi_{3\text{-HS-No-Auth}}, \mathcal{A}, 1}$.

$\mathbf{H}_0$ and $\mathbf{H}_1$ are indistinguishable: In $\mathbf{H}_1$, w_2 is uniformly random by the randomness of q_0 and w_3 is uniformly random by the randomness of $\mathbf{r}$. P_1 has no ability to cheat in the invocation of **Create** and **Open**, by the properties of $\mathcal{F}_{\text{AuSS}}$. Therefore $w_2 + w_3 = q_0 + \sum \mathbf{q}[i] \mathbf{v}[i]$, and the check in step 4 will fail if and only if $\mathcal{A}$ provides a $w^* \neq w_2 + w_3$, in which case **Open** will return $w^* - w_2 - w_3$, as in $\mathbf{H}_0$.

$\mathbf{H}_1$ and $\mathbf{H}_2$ are indistinguishable, because the expression in Step 8 can be written as:

$$\sum_{i=1}^n \mathbf{z}[i] \cdot (\mathbf{v}[i] - \pi^{-1}(\mathbf{v}^\pi[i]),$$

which will be nonzero with overwhelming probability if $\pi^{-1}(\mathbf{v}^\pi) \neq \mathbf{v}$, by the randomness of $\mathbf{z}$. In particular, if $\mathcal{A}$ chooses $[\mathbf{v}^\pi]_1^*$ such that $[\mathbf{v}^\pi]_1^* \neq [\mathbf{v}^\pi]_1$, we have $\pi^{-1}(\mathbf{v}^\pi) \neq \mathbf{v}$.

We prove security against a malicious P_2 below; security against P_3 is similar.

Lemma 6. *Protocol 4 is secure against malicious P_2 .*

Proof. The simulator $\mathcal{S}$ interacts with the ideal functionality and the adversary $\mathcal{A}$ as follows:

1. Extract $\mathcal{A}$'s input $\mathbf{r}^*$ to **Create** in Step 1, generates q_0 and $\mathbf{q}$ randomly, and aborts in Step 4 if $\mathcal{A}$ sends any message $w_2^* \neq q_0 + \sum \mathbf{q}[i] \mathbf{r}^*[i]$ in Step 3.
2. Receive $\mathcal{A}$'s message $\mathbf{v}_{2,1}^*$ in Step 5, extract $\mathcal{A}$'s input $[\mathbf{v}^\pi]_2^*$ in Step 6, and abort in Step 8 if $\mathbf{v}_{2,1}^* + [\mathbf{v}^\pi]_2^* \neq \pi(\mathbf{r}^*)$.
3. Generate a random vector $\mathbf{z}$ and abort if $\mathcal{A}$ sends anything other than $\pi(\mathbf{z})$ to **Create** in Step 7.
4. Invoke the ideal functionality to obtain the authenticated output shares, sends this to $\mathcal{A}$, and outputs whatever $\mathcal{A}$ outputs.

Protocol 4 $\Pi_{3\text{-HS-No-Auth}}$: 3-Party Maliciously Secure Half-Shuffle without Authentication

Input: authenticated secret shares of vector, $\mathbf{v} \in \mathbb{F}^n$, $\langle \mathbf{v} \rangle$; P_2, P_3 with permutation, $\pi \leftarrow S_n$.

Output: $\langle \pi(\mathbf{v}) \rangle$, authenticated secret shares of $\pi(\mathbf{v})$.

- 1: P_2 samples $\mathbf{r} \xleftarrow{\$} \mathbb{F}^n$ and invokes **Create** from $\mathcal{F}_{\text{AuSS}}$ to distribute shares $\langle \mathbf{r} \rangle$ to the other two parties.
 - 2: The parties invoke **Scale**, then **Add** to generate an authenticated sharing of $\langle \mathbf{v} \rangle - \langle \mathbf{r} \rangle$ and then invoke **Open** to allow P_3 to learn the resulting value $\mathbf{v} - \mathbf{r}$. Writing $\mathbf{v}'_2 = \mathbf{r}$ and $\mathbf{v}'_3 = \mathbf{v} - \mathbf{r}$, note that this gives a two party (un-authenticated) additive sharing of $\mathbf{v}$.
 - 3: P_2 and P_3 obtain a random value q_0 and a random vector $\mathbf{q} \in \mathbb{F}^n$ from $\mathcal{F}_{\text{PREP}}$. P_2 sends $w_2 := q_0 + \sum \mathbf{q}[i]\mathbf{v}'_2[i]$ to P_1 , and P_3 sends $w_3 := \sum \mathbf{q}[i]\mathbf{v}'_3[i]$ to P_1 .
 - 4: P_1 invokes **Create** from $\mathcal{F}_{\text{AuSS}}$ to secret share $\langle w \rangle := w_2 + w_3$. Then all parties **Open** $\mathbf{q}$, and compute and **Open** $\langle w \rangle - \langle q_0 \rangle - \sum_{i=1}^n \mathbf{q}[i] \cdot \langle \mathbf{v}[i] \rangle$ and abort if the result is nonzero.
 - 5: For $j = 2, 3$, party P_j samples $\mathbf{r}_j \xleftarrow{\$} \mathbb{F}^n$, and sends $\mathbf{v}_{j,1} := \pi(\llbracket \mathbf{v}' \rrbracket_j) - \mathbf{r}_j$ to P_1 .
 - 6: P_1 sets their share $[\mathbf{v}^\pi]_1 := \mathbf{v}_{2,1} + \mathbf{v}_{3,1}$. For $j = 2, 3$ P_j sets their share $[\mathbf{v}^\pi]_j := \mathbf{r}_j$. All parties invoke **Create** from $\mathcal{F}_{\text{AuSS}}$ to convert their unauthenticated shares into authenticated shares $\langle \mathbf{v}^\pi \rangle$.
 - 7: The parties obtain a random vector $\langle \mathbf{z} \rangle$ from $\mathcal{F}_{\text{PREP}}$, and open $\mathbf{z}$ to P_2 and P_3 . P_2 invokes **Create** from $\mathcal{F}_{\text{AuSS}}$ to share the vector $\mathbf{z}^\pi := \pi(\mathbf{z})$, and **Open** to open it to P_3 . If the opened value does not match P_3 's own computation of $\pi(\mathbf{z})$, P_3 aborts.
 - 8: The parties compute the authenticated secret share, $\sum_{i=1}^n \langle \mathbf{v}[i] \rangle \cdot \langle \mathbf{z}[i] \rangle - \langle \mathbf{v}^\pi[i] \rangle \cdot \langle \mathbf{z}^\pi[i] \rangle$, invoke **Check** to ensure it is well-formed, and then invoke **Open** and abort if the result is nonzero.
-

The only messages P_2 receives are mediated by $\mathcal{F}_{\text{AuSS}}$, so the transcripts in the ideal and real world are identical as long as both protocols do not abort. To establish indistinguishability, we must show that the protocols cannot be distinguished by their aborts, and that when the protocol does not abort, the other parties hold correct shares.

The real world protocol can abort in Step 4, Step 7, and Step 8. These aborts correspond to Steps 1, 3, and 2, respectively of our simulator. In Step 4 of the real world protocol, P_1 invokes **Create** on

$$w_2^* + w_3 := w_2^* + \sum \mathbf{q}[i]\mathbf{v}[i] - \sum \mathbf{q}[i]\mathbf{r}^*[i],$$

and the parties then open $w_2^* + w_3 - q_0 - \sum \mathbf{q}[i]\mathbf{v}[i] = w_2^* - q_0 - \sum \mathbf{q}[i]\mathbf{r}^*$, so aborts in the real world exactly when the simulator aborts in its simulated Step 1. In Step 7 of the real world protocol, P_3 will abort unless they receive $\pi(\mathbf{z})$ when **Open** is invoked on $\mathbf{z}^\pi$. Under the assumption that P_2 and P_3 have the same input π , this is exactly equal to the simulated Step 3. In Step 8 of the real world protocol, as in the proof of security against P_1 , we can rewrite the tested expression as $\sum_{i=1}^n \mathbf{z}[i] \cdot (\mathbf{v}[i] - \pi^{-1}(\mathbf{v}^\pi[i]))$, which will be nonzero with overwhelming probability if $\mathbf{v}[i] \neq \pi^{-1}(\mathbf{v}^\pi[i])$, since we obtain the randomly sampled $\mathbf{z}$ after all parties have committed to their share of $\mathbf{v}^\pi$ via **Create**. The

sum of the shares of P_1 and P_3 is $\mathbf{v}_{2,1}^* + \mathbf{v}_{3,1} + \mathbf{r}_3 = \mathbf{v}_{2,1}^* + \pi(\mathbf{v} - \mathbf{r}^*)$, so the share that P_2 must supply in the real world to avoid an abort is $[\mathbf{v}^\pi]_2^* = \pi(\mathbf{r}^*) - \mathbf{v}_{2,1}^*$. This matches the condition in the simulated Step 2.

Acknowledgements

The authors thank Sela Navot for pointing out a potential vulnerability using malformed ciphertexts, and further helpful conversations. This research was supported in part by NSF grants CNS-2246355, CCF-2220450, US-Israel BSF grant 2022370, and by Sunday Group.

References

1. A, P.S., Koti, N., Kukkala, V.B., Patra, A., Gopal, B.R., Sangal, S.: Ruffle: Rapid 3-party shuffle protocols. *Proc. Priv. Enhancing Technol.* **2023**(3), 24–42 (2023). <https://doi.org/10.56553/POPETS-2023-0068>, <https://doi.org/10.56553/popets-2023-0068>
2. Abraham, I., Pinkas, B., Yanai, A.: Blinder - scalable, robust anonymous committed broadcast. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security*, Virtual Event, USA, November 9–13, 2020. pp. 1233–1252. ACM (2020). <https://doi.org/10.1145/3372297.3417261>, <https://doi.org/10.1145/3372297.3417261>
3. Araki, T., Furukawa, J., Ohara, K., Pinkas, B., Rosemarin, H., Tsuchida, H.: Secure graph analysis at scale. In: Kim, Y., Kim, J., Vigna, G., Shi, E. (eds.) *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security*, Virtual Event, Republic of Korea, November 15 – 19, 2021. pp. 610–629. ACM (2021). <https://doi.org/10.1145/3460120.3484560>, <https://doi.org/10.1145/3460120.3484560>
4. Aranha, D.F., Costache, A., Guimarães, A., Soria-Vazquez, E.: Heliopolis: Verifiable computation over homomorphically encrypted data from interactive oracle proofs is practical. In: Chung, K.M., Sasaki, Y. (eds.) *Advances in Cryptology – ASIACRYPT 2024*. pp. 302–334. Springer Nature Singapore, Singapore (2025)
5. Asharov, G., Hamada, K., Ikarashi, D., Kikuchi, R., Nof, A., Pinkas, B., Takahashi, K., Tomida, J.: Efficient secure three-party sorting with applications to data analysis and heavy hitters. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7–11, 2022*. pp. 125–138. ACM (2022). <https://doi.org/10.1145/3548606.3560691>, <https://doi.org/10.1145/3548606.3560691>
6. Ben-Or, M., Goldwasser, S., Wigderson, A.: Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In: Simon, J. (ed.) *Proceedings of the 20th Annual ACM Symposium on Theory of Computing*, May 2–4, 1988, Chicago, Illinois, USA. pp. 1–10. ACM (1988). <https://doi.org/10.1145/62212.62213>, <https://doi.org/10.1145/62212.62213>
7. Bendlin, R., Damgård, I., Orlandi, C., Zakarias, S.: Semi-homomorphic encryption and multiparty computation. In: Paterson, K.G. (ed.) *Advances in Cryptology – EUROCRYPT 2011*. pp. 169–188. Springer Berlin Heidelberg, Berlin, Heidelberg (2011)

8. Bitansky, N., Chiesa, A., Ishai, Y., Ostrovsky, R., Paneth, O.: Succinct non-interactive arguments via linear interactive proofs. In: Sahai, A. (ed.) Theory of Cryptography - 10th Theory of Cryptography Conference, TCC 2013, Tokyo, Japan, March 3-6, 2013. Proceedings. Lecture Notes in Computer Science, vol. 7785, pp. 315–333. Springer (2013). https://doi.org/10.1007/978-3-642-36594-2_18, https://doi.org/10.1007/978-3-642-36594-2_18
9. Blunk, M., Bunn, P., Dittmer, S., Lu, S., Ostrovsky, R.: Secure merge in linear time and $o(\log \log N)$ rounds. IACR Cryptol. ePrint Arch. p. 590 (2022), <https://eprint.iacr.org/2022/590>
10. Bois, A., Cascudo, I., Fiore, D., Kim, D.: Flexible and efficient verifiable computation on encrypted data. In: Garay, J.A. (ed.) Public-Key Cryptography - PKC 2021 - 24th IACR International Conference on Practice and Theory of Public Key Cryptography, Virtual Event, May 10-13, 2021, Proceedings, Part II. Lecture Notes in Computer Science, vol. 12711, pp. 528–558. Springer (2021). https://doi.org/10.1007/978-3-030-75248-4_19, https://doi.org/10.1007/978-3-030-75248-4_19
11. Boneh, D., Ishai, Y., Sahai, A., Wu, D.J.: Lattice-based SNARGs and their application to more efficient obfuscation. In: Coron, J., Nielsen, J.B. (eds.) Advances in Cryptology - EUROCRYPT 2017 - 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Paris, France, April 30 - May 4, 2017, Proceedings, Part III. Lecture Notes in Computer Science, vol. 10212, pp. 247–277 (2017). https://doi.org/10.1007/978-3-319-56617-7_9, https://doi.org/10.1007/978-3-319-56617-7_9
12. Boneh, D., Ishai, Y., Sahai, A., Wu, D.J.: Quasi-optimal SNARGs via linear multi-prover interactive proofs. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part III. Lecture Notes in Computer Science, vol. 10822, pp. 222–255. Springer (2018). https://doi.org/10.1007/978-3-319-78372-7_8, https://doi.org/10.1007/978-3-319-78372-7_8
13. Brakerski, Z., Döttling, N., Garg, S., Malavolta, G.: Leveraging linear decryption: Rate-1 fully-homomorphic encryption and time-lock puzzles. In: Theory of Cryptography Conference. pp. 407–437. Springer (2019)
14. Cascudo, I., Costache, A., Cozzo, D., Fiore, D., Guimarães, A., Soria-Vazquez, E.: Verifiable computation for approximate homomorphic encryption schemes. In: Tauman Kalai, Y., Kamara, S.F. (eds.) Advances in Cryptology – CRYPTO 2025. pp. 643–677. Springer Nature Switzerland, Cham (2025)
15. Catalano, D., Fiore, D.: Practical homomorphic macs for arithmetic circuits. In: Johansson, T., Nguyen, P.Q. (eds.) Advances in Cryptology - EUROCRYPT 2013, 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26-30, 2013. Proceedings. Lecture Notes in Computer Science, vol. 7881, pp. 336–352. Springer (2013). https://doi.org/10.1007/978-3-642-38348-9_21, https://doi.org/10.1007/978-3-642-38348-9_21
16. Chakraborty, S., Peceny, S., Raghuraman, S., Rindal, P.: Logstar: Efficient linear* time secure merge. IACR Cryptol. ePrint Arch. p. 159 (2024), <https://eprint.iacr.org/2024/159>
17. Chase, M., Ghosh, E., Poburinnaya, O.: Secret-shared shuffle. In: Moriai, S., Wang, H. (eds.) Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part III. Lecture Notes

- in Computer Science, vol. 12493, pp. 342–372. Springer (2020). https://doi.org/10.1007/978-3-030-64840-4_12, https://doi.org/10.1007/978-3-030-64840-4_12
18. Chatel, S., Knabenhans, C., Pyrgelis, A., Troncoso, C., Hubaux, J.: VERITAS: plaintext encoders for practical verifiable homomorphic encryption. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024. pp. 2520–2534. ACM (2024). <https://doi.org/10.1145/3658644.3670282>, <https://doi.org/10.1145/3658644.3670282>
 19. Cho, C., Dittmer, S., Ishai, Y., Lu, S., Ostrovsky, R.: Rabbit-mix: Robust algebraic anonymous broadcast from additive bases. In: Balzarotti, D., Xu, W. (eds.) 33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024. USENIX Association (2024), <https://www.usenix.org/conference/usenixsecurity24/presentation/cho-chongwon>
 20. Chung, H., Kim, D., Kim, J.H., Kim, J.: Amortized efficient zk-snark from linear-only RLWE encodings. J. Commun. Networks **25**(3), 271–284 (2023). <https://doi.org/10.23919/JCN.2023.000012>, <https://doi.org/10.23919/jcn.2023.000012>
 21. Damgård, I., Keller, M., Larraia, E., Pastro, V., Scholl, P., Smart, N.P.: Practical covertly secure mpc for dishonest majority – or: Breaking the spdz limits. In: Crampton, J., Jajodia, S., Mayes, K. (eds.) Computer Security – ESORICS 2013. pp. 1–18. Springer Berlin Heidelberg, Berlin, Heidelberg (2013)
 22. Damgård, I., Pastro, V., Smart, N.P., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. In: Safavi-Naini, R., Canetti, R. (eds.) Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings. Lecture Notes in Computer Science, vol. 7417, pp. 643–662. Springer (2012). https://doi.org/10.1007/978-3-642-32009-5_38, https://doi.org/10.1007/978-3-642-32009-5_38
 23. Di, Z., Xia, L., Nguyen, W.D., Tyagi, N.: Muxproofs: Succinct arguments for machine computation from vector lookups. In: Chung, K., Sasaki, Y. (eds.) Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part V. Lecture Notes in Computer Science, vol. 15488, pp. 236–265. Springer (2024). https://doi.org/10.1007/978-981-96-0935-2_8, https://doi.org/10.1007/978-981-96-0935-2_8
 24. Eskandarian, S., Boneh, D.: Clarion: Anonymous communication from multiparty shuffling protocols. In: 29th Annual Network and Distributed System Security Symposium, NDSS 2022, San Diego, California, USA, April 24-28, 2022. The Internet Society (2022), <https://www.ndss-symposium.org/ndss-paper/auto-draft-243/>
 25. Evans, D., Kolesnikov, V., Rosulek, M.: A pragmatic introduction to secure multiparty computation. Found. Trends Priv. Secur. **2**(2-3), 70–246 (2018). <https://doi.org/10.1561/33000000019>, <https://doi.org/10.1561/33000000019>
 26. Falk, B.H., Noble, D., Ostrovsky, R.: Alibi: A flaw in cuckoo-hashing based hierarchical ORAM schemes and a solution. In: Canteaut, A., Standaert, F. (eds.) Advances in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17-21, 2021, Proceedings, Part III. Lecture Notes in Computer Science, vol. 12698, pp. 338–369. Springer (2021). https://doi.org/10.1007/978-3-030-77883-5_12, https://doi.org/10.1007/978-3-030-77883-5_12
 27. Falk, B.H., Ostrovsky, R.: Secure merge with $o(n \log \log n)$ secure operations. In: Tessaro, S. (ed.) 2nd Conference on Information-Theoretic Cryptography, ITC 2021, July 23-26, 2021, Virtual Conference. LIPIcs, vol. 199, pp. 7:1–7:29.

- Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2021). <https://doi.org/10.4230/LIPICS.ITC.2021.7>, <https://doi.org/10.4230/LIPICS.ITC.2021.7>
28. Fiore, D., Nitulescu, A., Pointcheval, D.: Boosting verifiable computation on encrypted data. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) *Public-Key Cryptography - PKC 2020 - 23rd IACR International Conference on Practice and Theory of Public-Key Cryptography*, Edinburgh, UK, May 4-7, 2020, Proceedings, Part II. *Lecture Notes in Computer Science*, vol. 12111, pp. 124–154. Springer (2020). https://doi.org/10.1007/978-3-030-45388-6_5, https://doi.org/10.1007/978-3-030-45388-6_5
 29. Fisher, R.A., Yates, F.: *Statistical Tables for Biological, Agricultural and Medical Research*. Oliver and Boyd, London, 1st edn. (1938)
 30. Franzese, N., Katz, J., Lu, S., Ostrovsky, R., Wang, X., Weng, C.: Constant-overhead zero-knowledge for RAM programs. In: Kim, Y., Kim, J., Vigna, G., Shi, E. (eds.) *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security*, Virtual Event, Republic of Korea, November 15 - 19, 2021. pp. 178–191. ACM (2021). <https://doi.org/10.1145/3460120.3484800>, <https://doi.org/10.1145/3460120.3484800>
 31. Ganesh, C., Nitulescu, A., Soria-Vazquez, E.: Rinocchio: Snarks for ring arithmetic. *J. Cryptol.* **36**(4), 41 (2023). <https://doi.org/10.1007/S00145-023-09481-3>, <https://doi.org/10.1007/s00145-023-09481-3>
 32. Ganesh, C., Khovratovich, D., Orsini, L., Boneh, D., Scholl, P.: Rinocchio: SNARKs for ring arithmetic. In: 30th USENIX Security Symposium (USENIX Security 21). pp. 2553–2570. USENIX Association (2021), <https://www.usenix.org/conference/usenixsecurity21/presentation/ganesh>
 33. Gennaro, R., Minelli, M., Nitulescu, A., Orru, M.: Lattice-based zk-SNARKs from square span programs. In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. pp. 556–573. ACM (2018). <https://doi.org/10.1145/3243734.3243766>
 34. Gennaro, R., Wichs, D.: Fully homomorphic message authenticators. In: Sako, K., Sarkar, P. (eds.) *Advances in Cryptology - ASIACRYPT 2013 - 19th International Conference on the Theory and Application of Cryptology and Information Security*, Bengaluru, India, December 1-5, 2013, Proceedings, Part II. *Lecture Notes in Computer Science*, vol. 8270, pp. 301–320. Springer (2013). https://doi.org/10.1007/978-3-642-42045-0_16, https://doi.org/10.1007/978-3-642-42045-0_16
 35. Gentry, C., Halevi, S., Jutla, C.S., Raykova, M.: Private database access with he-over-oram architecture. In: Malkin, T., Kolesnikov, V., Lewko, A.B., Polychronakis, M. (eds.) *Applied Cryptography and Network Security - 13th International Conference, ACNS 2015, New York, NY, USA, June 2-5, 2015, Revised Selected Papers. Lecture Notes in Computer Science*, vol. 9092, pp. 172–191. Springer (2015). https://doi.org/10.1007/978-3-319-28166-7_9, https://doi.org/10.1007/978-3-319-28166-7_9
 36. Goldreich, O.: *The Foundations of Cryptography - Volume 1: Basic Techniques*. Cambridge University Press (2001). <https://doi.org/10.1017/CBO9780511546891>, <http://www.wisdom.weizmann.ac.il/%7Eoded/foc-vol1.html>
 37. Goldreich, O., Micali, S., Wigderson, A.: How to play any mental game or A completeness theorem for protocols with honest majority. In: Aho, A.V. (ed.) *Proceedings of the 19th Annual ACM Symposium on Theory of Computing*, 1987, New York, New York, USA. pp. 218–229. ACM (1987). <https://doi.org/10.1145/28395.28420>, <https://doi.org/10.1145/28395.28420>

38. Goldreich, O., Ostrovsky, R.: Software protection and simulation on oblivious rams. *J. ACM* **43**(3), 431–473 (1996). <https://doi.org/10.1145/233551.233553>, <https://doi.org/10.1145/233551.233553>
39. Hamada, K., Ikarashi, D., Chida, K., Takahashi, K.: Oblivious radix sort: An efficient sorting algorithm for practical secure multi-party computation. *IACR Cryptol. ePrint Arch.* p. 121 (2014), <http://eprint.iacr.org/2014/121>
40. Hamada, K., Kikuchi, R., Ikarashi, D., Chida, K., Takahashi, K.: Practically efficient multi-party sorting protocols from comparison sort algorithms. In: Kwon, T., Lee, M., Kwon, D. (eds.) *Information Security and Cryptology - ICISC 2012 - 15th International Conference*, Seoul, Korea, November 28–30, 2012, Revised Selected Papers. *Lecture Notes in Computer Science*, vol. 7839, pp. 202–216. Springer (2012). https://doi.org/10.1007/978-3-642-37682-5_15, https://doi.org/10.1007/978-3-642-37682-5_15
41. Hasler, S., Reiser, P., Rivinius, M., Küsters, R.: Multipars: Reduced-communication mpc over $\mathbb{Z}_{2^k}$. *Proceedings on Privacy Enhancing Technologies (PoPETs)* **2024**(1), 5–28 (2024). <https://doi.org/10.56553/popets-2024-0038>, <https://petsymposium.org/popets/2024/popets-2024-0038.pdf>
42. Hemenway, B., Libert, B., Ostrovsky, R., Vergnaud, D.: Lossy encryption: Constructions from general assumptions and efficient selective opening chosen ciphertext security. In: Lee, D.H., Wang, X. (eds.) *Advances in Cryptology – ASIACRYPT 2011*. pp. 70–88. Springer Berlin Heidelberg, Berlin, Heidelberg (2011)
43. Huang, Y., Evans, D., Katz, J.: Private set intersection: Are garbled circuits better than custom protocols? In: 19th Annual Network and Distributed System Security Symposium, NDSS 2012, San Diego, California, USA, February 5–8, 2012. The Internet Society (2012), <https://www.ndss-symposium.org/ndss2012/private-set-intersection-are-garbled-circuits-better-custom-protocols>
44. Ishai, Y., Su, H., Wu, D.J.: Shorter and faster post-quantum designated-verifier zkSNARKs from lattices. In: Kim, Y., Kim, J., Vigna, G., Shi, E. (eds.) *CCS ’21: 2021 ACM SIGSAC Conference on Computer and Communications Security*, Virtual Event, Republic of Korea, November 15 – 19, 2021. pp. 212–234. ACM (2021). <https://doi.org/10.1145/3460120.3484572>, <https://doi.org/10.1145/3460120.3484572>
45. Keller, M.: Mp-spdz: A versatile framework for multi-party computation. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. pp. 1575–1590. CCS ’20, Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3372297.3417872>, <https://doi.org/10.1145/3372297.3417872>
46. Keller, M., Orsini, E., Scholl, P.: Actively secure OT extension with optimal overhead. In: Gennaro, R., Robshaw, M. (eds.) *Advances in Cryptology – CRYPTO 2015*. pp. 724–741. Springer Berlin Heidelberg, Berlin, Heidelberg (2015)
47. Keller, M., Orsini, E., Scholl, P.: Mascot: Faster malicious arithmetic secure computation with oblivious transfer. In: *Proceedings of the 23rd ACM SIGSAC Conference on Computer and Communications Security*. pp. 830–842 (2016). <https://doi.org/10.1145/2976749.2978357>, <https://dl.acm.org/doi/10.1145/2976749.2978357>
48. Knabenhans, C., Viand, A., Merino-Gallardo, A., Hithnawi, A.: vfhe: Verifiable fully homomorphic encryption. In: Bergamaschi, F., Costache, A., Rohloff, K. (eds.) *Proceedings of the 12th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, Salt Lake City, UT, USA, October 14–18, 2024. pp. 11–22. ACM (2024). <https://doi.org/10.1145/3689945.3694806>, <https://doi.org/10.1145/3689945.3694806>
49. Knuth, D.E.: *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3rd edn. (1997)

50. Larsen, K.G., Nielsen, J.B.: Yes, there is an oblivious ram lower bound! In: Advances in Cryptology – CRYPTO 2018. pp. 523–542. Springer International Publishing, Cham (2018)
51. Laud, P.: Linear-time oblivious permutations for SPDZ. In: Conti, M., Stevens, M., Krenn, S. (eds.) Cryptology and Network Security - 20th International Conference, CANS 2021, Vienna, Austria, December 13–15, 2021, Proceedings. Lecture Notes in Computer Science, vol. 13099, pp. 245–252. Springer (2021). https://doi.org/10.1007/978-3-030-92548-2_13, https://doi.org/10.1007/978-3-030-92548-2_13
52. Li, S., Wang, X., Zhang, R.: Privacy-preserving homomorphic macs with efficient verification. In: Jin, H., Wang, Q., Zhang, L. (eds.) Web Services - ICWS 2018 - 25th International Conference, Held as Part of the Services Conference Federation, SCF 2018, Seattle, WA, USA, June 25–30, 2018, Proceedings. Lecture Notes in Computer Science, vol. 10966, pp. 100–115. Springer (2018). https://doi.org/10.1007/978-3-319-94289-6_7, https://doi.org/10.1007/978-3-319-94289-6_7
53. Lindell, Y.: How To Simulate It — A Tutorial on the Simulation Proof Technique. In: Information Security and Cryptography, pp. 277–346. Springer International Publishing (2017). https://doi.org/10.1007/978-3-319-57048-8_6, https://doi.org/10.1007/978-3-319-57048-8_6
54. Liu, H., Wang, X., Yang, K., Yu, Y.: Authenticated bitgc for actively secure rate-one 2pc. IACR Cryptol. ePrint Arch. p. 232 (2025), <https://eprint.iacr.org/2025/232>
55. Lu, S., Ostrovsky, R.: Distributed oblivious RAM for secure two-party computation. In: Sahai, A. (ed.) Theory of Cryptography - 10th Theory of Cryptography Conference, TCC 2013, Tokyo, Japan, March 3–6, 2013. Proceedings. Lecture Notes in Computer Science, vol. 7785, pp. 377–396. Springer (2013). https://doi.org/10.1007/978-3-642-36594-2_22, https://doi.org/10.1007/978-3-642-36594-2_22
56. Motwani, R., Raghavan, P.: Randomized Algorithms. Cambridge University Press (1995)
57. Noble, D., Hemenway Falk, B., Ostrovsky, R.: Metadoram: Breaking the log-overhead information theoretic barrier. In: 6th Conference on Information-Theoretic Cryptography (ITC 2025). Leibniz International Proceedings in Informatics (LIPIcs) (August 2025), accepted; forthcoming
58. Orsini, E., Smart, N.P., Vercauteren, F.: Overdrive2k: Efficient secure mpc over $\mathbb{Z}_{2^k}$ from somewhat homomorphic encryption. In: Topics in Cryptology – CT-RSA 2020. Lecture Notes in Computer Science, vol. 12100, pp. 254–283. Springer (2020). https://doi.org/10.1007/978-3-030-40186-3_12, https://dl.acm.org/doi/10.1007/978-3-030-40186-3_12
59. Ostrovsky, B.: Privacy-preserving dijkstra. In: Advances in Cryptology – CRYPTO 2024, Part IX. Lecture Notes in Computer Science, Springer (2024). https://doi.org/10.1007/978-3-031-68400-5_3
60. Ostrovsky, R., Skeith III, W.E.: A survey of single-database private information retrieval: Techniques and applications. In: International Workshop on Public Key Cryptography. pp. 393–411. Springer (2007)
61. Song, X., Yin, D., Bai, J., Dong, C., Chang, E.: Secret-shared shuffle with malicious security. In: 31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024. The Internet Society (2024), <https://www.ndss-symposium.org/ndss-paper/secret-shared-shuffle-with-malicious-security/>
62. Stefanov, E., van Dijk, M., Shi, E., Chan, T.H., Fletcher, C.W., Ren, L., Yu, X., Devadas, S.: Path ORAM: an extremely simple oblivious RAM protocol. J.

- ACM **65**(4), 18:1–18:26 (2018). <https://doi.org/10.1145/3177872>, <https://doi.org/10.1145/3177872>
63. Ulukus, S., Avestimehr, S., Gastpar, M., Jafar, S.A., Tandon, R., Tian, C.: Private retrieval, computing, and learning: Recent progress and future challenges. *IEEE J. Sel. Areas Commun.* **40**(3), 729–748 (2022). <https://doi.org/10.1109/JSAC.2022.3142358>, <https://doi.org/10.1109/JSAC.2022.3142358>
 64. Waksman, A.: A permutation network. *J. ACM* **15**(1), 159–163 (1968). <https://doi.org/10.1145/321439.321449>, <https://doi.org/10.1145/321439.321449>
 65. Wang, X., Ranellucci, S., Katz, J.: Authenticated garbling and efficient maliciously secure two-party computation. In: Thuraisingham, B., Evans, D., Malkin, T., Xu, D. (eds.) *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*. pp. 21–37. ACM (2017). <https://doi.org/10.1145/3133956.3134053>, <https://doi.org/10.1145/3133956.3134053>
 66. Yao, A.C.: How to generate and exchange secrets (extended abstract). In: *27th Annual Symposium on Foundations of Computer Science, Toronto, Canada, 27-29 October 1986*. pp. 162–167. IEEE Computer Society (1986). <https://doi.org/10.1109/SFCS.1986.25>, <https://doi.org/10.1109/SFCS.1986.25>

A Security Proofs for $\Pi_{3\text{-HS}}$ (Protocol 2)

We prove security of $\Pi_{3\text{-HS}}$ in the following lemmas.

Lemma 7. *Protocol 2 is secure against malicious P_1 in the $\mathcal{F}_{\text{AuSS}}$ -hybrid model.*

Proof. The proof follows directly from the security of $\mathcal{F}_{\text{AuSS}}$. Informally, the only way P_1 can cheat is by using incorrect shares in calls to **ReShare**. But by the definition of $\mathcal{F}_{\text{AuSS}}$, the functionality aborts in this case. Formally, the simulator $\mathcal{S}_{3\text{-HS}}^1$ interacts with the ideal functionality $\mathcal{F}_{3\text{-HS}}$ and the adversary $\mathcal{A}$ as follows:

1. Receive share from $\mathcal{A}$. If it's not equal to $\langle \mathbf{v} \rangle_1$, $\mathcal{A}$'s input, abort.
2. Else, invoke $\mathcal{F}_{3\text{-HS}}$ with share $\langle \mathbf{v} \rangle_1$ as P_1 's input and receive output share $\langle \mathbf{v}^\pi \rangle_1$.
3. Send $\langle \mathbf{v}^\pi \rangle_1$ to $\mathcal{A}$.

Indistinguishability follows from the security of $\mathcal{F}_{\text{AuSS}}$. In particular, **ReShare** always outputs valid authenticated secret shares which are uniformly random in the view of $\mathcal{A}$ by the security of $\mathcal{F}_{\text{AuSS}}$. The output of $\mathcal{F}_{3\text{-HS}}$ is also a uniformly random share and therefore, $\mathcal{A}$'s received message in both the ideal and real world is identically distributed. Moreover, by the correctness of $\Pi_{3\text{-HS}}$, $\langle \mathbf{v}^\pi \rangle$ is indeed a valid AuSS of $\pi(\mathbf{v})$ in the real world; and in the ideal world, $\mathcal{A}$ receives a share that is consistent with the honest parties' output. That is, collectively they hold an AuSS of the same value, $\pi(\mathbf{v})$. Therefore, the ideal execution is identically distributed to the real execution.

We prove security against a malicious P_2 below. Security against a malicious P_3 is identical.

Lemma 8. *Protocol 2 is secure against malicious P_2 .*

Proof. Again, security follows almost directly. Formally, we construct a simulator $\mathcal{S}_{3\text{-HS}}^2$ that interacts with the ideal functionality $\mathcal{F}_{3\text{-HS}}$ and the adversary $\mathcal{A}$ as follows:

1. Sample a uniformly value r from the domain of AuSS and send it to $\mathcal{A}$.
2. If $\mathcal{A}$ sends a permutation π' . If $\pi' \neq \pi$, $\mathcal{A}$'s input, then abort.
3. Else invoke $\mathcal{F}_{3\text{-HS}}$ with π as P_2 's input and receive output share $\langle \mathbf{v}^\pi \rangle_2$.
4. Send $\langle \mathbf{v}^\pi \rangle_2$ to $\mathcal{A}$.

In the real execution, $\mathcal{A}$ receives as input a share from **ReShare** which appears uniformly random in the view of $\mathcal{A}$ by the security of $\mathcal{F}_{\text{AuSS}}$. Thus, $\mathcal{S}_{3\text{-HS}}^2$ simulates perfectly in the ideal execution using step 1 of the simulation. If $\mathcal{A}$ uses the wrong permutation, then $\mathcal{A}$ and P_3 disagree about the output sharing and by definition of **ReShare**, $\mathcal{F}_{\text{AuSS}}$ aborts, which is identical to $\mathcal{S}_{3\text{-HS}}^2$'s abort in step 2. Finally, $\mathcal{A}$ receives an output share $\langle \mathbf{v}^\pi \rangle_2$ that is identically distributed in both the ideal and real world by the correctness of $\Pi_{3\text{-HS}}$. We conclude the ideal execution is identically distributed to the real execution.

B Concrete Implementation with SPDZ AuSS

We present our two-party and three-party half-shuffle protocols (Protocols 2 and 3) when instantiated with the SPDZ AuSS as described in Section 3.1.

Protocol 5 $\Pi_{3\text{-HS-SPDZ}}$: 3-Party Maliciously Secure Half-Shuffle with SPDZ AuSS

Input: SPDZ AuSS of vector, $\mathbf{v} \in \mathbb{F}^n$, $\langle \mathbf{v} \rangle$; P_2, P_3 with permutation, $\pi \leftarrow S_n$.

Output: $\langle \pi(\mathbf{v}) \rangle$, SPDZ AuSS of permuted vector $\pi(\mathbf{v})$.

- 1: P_1 samples $\mathbf{r}, \mathbf{s} \xleftarrow{\$} \mathbb{F}^n$ and sends $\langle \mathbf{v} \rangle_{1,2} := (\mathbf{r}, \mathbf{s})$ to P_2 , and $\langle \mathbf{v} \rangle_{1,3} := (\llbracket \mathbf{v} \rrbracket_1 - \mathbf{r}, \llbracket \gamma(\mathbf{v}) \rrbracket_1 - \mathbf{s})$ to P_3 .
 - 2: The three parties invoke **Check** as described in Section 3.1 on $(\langle \mathbf{v} \rangle_{1,2}, \langle \mathbf{v} \rangle_{1,3})$ and abort if the check fails.
 - 3: For $j = 2, 3$, party P_j samples $\mathbf{r}_j, \mathbf{s}_j \xleftarrow{\$} \mathbb{F}^n$, and sends $\langle \mathbf{m} \rangle_{j,1} := (\pi(\llbracket \mathbf{v} \rrbracket_{1,j}) + \mathbf{r}_j, \pi(\llbracket \gamma(\mathbf{v}) \rrbracket_{1,j}) + \mathbf{s}_j)$ to P_1 .
 - 4: P_1 sets their share $\langle \mathbf{v}^\pi \rangle_1 := \langle \mathbf{m} \rangle_{2,1} + \langle \mathbf{m} \rangle_{3,1}$.
 - 5: For $j = 2, 3$, party P_j sets their share $\langle \mathbf{v}^\pi \rangle_j := \pi(\langle \mathbf{v} \rangle_j) - (\mathbf{r}_j, \mathbf{s}_j)$.
 - 6: The three parties invoke **Check** on $\langle \mathbf{v}^\pi \rangle$ and abort if this check fails.
-

Protocol 6 $\Pi_{2\text{-HS-SPDZ}}$: 2-Party Maliciously Secure Half-Shuffle with SPDZ AuSS

Input: $\langle \mathbf{v} \rangle$, SPDZ AuSS of $\mathbf{v} \in \mathbb{F}^n$; P_2 with permutation, $\pi \leftarrow S_n$.

Output: $\langle \pi(\mathbf{v}) \rangle$, SPDZ AuSS of $\pi(\mathbf{v})$.

Part 1: commit to permutation

- 1: Use SPDZ preprocessing to generate $\langle \mathbf{z} \rangle$ for $\mathbf{z} \xleftarrow{\$} \mathbb{F}$. Open $\llbracket \mathbf{z} \rrbracket$ to P_2 .
- 2: P_2 computes $\mathbf{z}^\pi := \pi(\mathbf{z})$ and creates an authenticated secret sharing $\langle \mathbf{z}^\pi \rangle$ using **Create** as described in Section 3.1.

Part 2: split and encrypt shares

- 3: P_1 generates two LHE key-pairs using independent, fresh randomness, $(pk_1, sk_1), (pk_2, sk_2) \leftarrow \text{LHE.KeyGen}(1^\kappa)$ and publishes pk_1 and pk_2 .
 - 4: Use SPDZ preprocessing to generate $\langle \mathbf{m} \rangle$ for $\mathbf{m} \xleftarrow{\$} \mathbb{F}^n$.
 - 5: Compute $\langle \mathbf{a} \rangle \leftarrow \langle \mathbf{v} \rangle + \langle \mathbf{m} \rangle$.
-

(Protocol 6 continued.)

6: P_1 encrypts its shares,

$$C_a \leftarrow \text{LHE.Enc}(pk_1, \langle a \rangle_1), \text{ and } C_m \leftarrow \text{LHE.Enc}(pk_2, \langle m \rangle_1)$$

and sends C_a, C_m to P_2 .

- 7: P_2 checks the validity of C_a and C_m : $\text{isValid}(pk_1, C_a) \stackrel{?}{=} 1$ and $\text{isValid}(pk_2, C_m) \stackrel{?}{=} 1$. Abort if any invalid.
- 8: Use SPDZ preprocessing to generate $\langle q_1 \rangle, \langle q_2 \rangle$ for $q_1, q_2 \xleftarrow{\$} \mathbb{F}^n$. Open $\llbracket q_1 \rrbracket, \llbracket q_2 \rrbracket$ to P_2 .
- 9: P_2 homomorphically computes, $C_{w,1} \leftarrow \sum_{i=1}^n q_1[i] \cdot C_a[i]$, and $C_{w,2} \leftarrow \sum_{i=1}^n q_2[i] \cdot C_m[i]$ and sends $C_{w,1}, C_{w,2}$ to P_1 .
- 10: P_1 decrypts $C_{w,1}, C_{w,2}$ and sets, $\langle w_i \rangle_1 := \text{LHE.Dec}(pk_i, C_{w,i})$ for $i = 1, 2$.
- 11: P_2 sets, $\langle w_1 \rangle_2 := \sum_{i=1}^n q_1[i] \cdot \langle a[i] \rangle_2$, $\langle w_2 \rangle_2 := \sum_{i=1}^n q_2[i] \cdot \langle m[i] \rangle_2$.
- 12: Compute, $\langle p_1 \rangle \leftarrow \langle w_1 \rangle - \sum_{i=1}^n \langle q_1[i] \rangle \cdot \langle a[i] \rangle$, and $\langle p_2 \rangle \leftarrow \langle w_2 \rangle - \sum_{i=1}^n \langle q_2[i] \rangle \cdot \langle m[i] \rangle$. Open $\llbracket p_1 \rrbracket$ and $\llbracket p_2 \rrbracket$. Abort if any value is non-zero.

Part 3: permute and rerandomize encrypted shares

13: P_2 samples $r_1, r_2, s_1, s_2 \xleftarrow{\$} \mathbb{F}^n$ and homomorphically computes,

$$C_a^\pi \leftarrow \pi(C_a) \odot \text{LHE.Enc}(pk_1, (r_1, r_2)) \quad \text{and} \quad C_m^\pi \leftarrow \pi(C_m) \odot \text{LHE.Enc}(pk_2, (s_1, s_2))$$

Send C_a^π, C_m^π to P_1 .

- 14: P_2 sets $\langle a^\pi \rangle_2 \leftarrow \pi(\langle a \rangle_2) - (r_1, r_2)$, and $\langle m^\pi \rangle_2 \leftarrow \pi(\langle m \rangle_2) - (s_1, s_2)$.
- 15: P_1 decrypts and sets shares, $\langle a^\pi \rangle_1 \leftarrow \text{LHE.Dec}(sk_1, C_a^\pi)$, and $\langle m^\pi \rangle_1 \leftarrow \text{LHE.Dec}(sk_2, C_m^\pi)$.

Part 4: perform consistency checks

16: Jointly generate random nonzero weights $c_1, c_2, c_3 \xleftarrow{\$} \mathbb{F}$ from SPDZ preprocessing and execute the four set equality checks:

$$\begin{aligned} (1) \quad \langle b_1 \rangle &\leftarrow \mathcal{F}_{\text{set-eq}}(\langle a \rangle, \langle a^\pi \rangle), & (2) \quad \langle b_2 \rangle &\leftarrow \mathcal{F}_{\text{set-eq}}(\langle m \rangle, \langle m^\pi \rangle), \\ (3) \quad \langle b_3 \rangle &\leftarrow \mathcal{F}_{\text{set-eq}}(\langle z \rangle, \langle z^\pi \rangle), & (4) \quad \langle b_4 \rangle &\leftarrow \mathcal{F}_{\text{set-eq}}(\langle c_1 a + c_2 m + c_3 z \rangle, \langle c_1 a^\pi + c_2 m^\pi + c_3 z^\pi \rangle) \end{aligned}$$

Execute MAC check on each output. (These can be batched as in [22].) Abort if check fails. Partially open $\llbracket b_i \rrbracket$. Abort if any non-zero.

17: Parties create shares of the permuted vector by removing shares of mask,

$$\langle v^\pi \rangle \leftarrow \langle a^\pi \rangle - \langle m^\pi \rangle$$

18: Return $\langle v^\pi \rangle$.
